

# claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf\_ef85c152-821\agent-abbfd9b652d6efc8b.jsonl`
- SHA-256: `152efe740ea08d8c9f1d1a3869050a547a9aa0d1015d750591ce5c5268aa28ca`
- Source modified: `2026-06-16T06:30:51+00:00`
- Imported at: `2026-07-05T16:48:29+00:00`
- project: `wf\_ef85c152-821`
- session\_id: `068560d3-1ca2-4387-981a-d2d6bca55414`

## Transcript

### 1. user (2026-06-16T06:26:59.414Z)

LENS = PRIVACY / de-attribution. Can freeze\_real\_fixture leak attributable data? Is \_scan\_forbidden COMPLETE ? does it catch: a caller-supplied identifying slug, an identifying source\_note, fps/frame\_width as a venue/camera quasi-identifier, and (the hard one) the raw shuttle (x,y) trajectory coordinates that could correlate-back to a source video (de Montjoye fingerprinting)? Is the random fx\_slug truly non-reversible and is the surrogate->source map kept off-git? Does the refusal gate have any bypass (e.g. truthy-but-not-True, or skipping it)? Are minors/biometric exclusions ENFORCED in code or only human-asserted flags ? and does the doc honestly bound the 'non-attributable only relative to the unpublished source' claim?

AUDIT the SHIPPED 'Golden Regression Fixtures' feature (4 merged PRs #186/#189/#191/#192) for any gap the per-PR verification missed.  
Read files in the git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-doc\ (master HEAD 8d3e185): backend/eval/golden\_fixtures.py (the core); tests/test\_golden\_fixtures.py; tests/test\_golden\_real\_fixtures.py; eval\_baselines/fixtures/ (3 synthetic fixtures + manifest.json + README.md); .github/workflows/ci.yml (the golden-crossos job); docs/GOLDEN\_REGRESSION\_FIXTURES.md (\$4c/\$5/\$6 claims). For analysis also read backend/eval/{experiment,distill\_local,metrics,gt\_loader,regression}.py and backend/pipeline/detectors/{tracknet\_runner,rally\_gate}.py.  
THE FEATURE: a video-free regression harness. replay\_fixture = parse\_trajectory\_csv -> distill\_local.build\_candidates(PINNED preset) -> VideoCandidates(5 shuttle cols only) -> experiment.predict(CONTROL, pure-stdlib) -> metrics.score. Synthetic Tier-0 fixtures + characterization (parse-compare: exact on int/structural, approx 1e-6 on float). freeze\_real\_fixture = de-attribution (random fx\_hex slug, metadata strip, metric-invariant time-origin reset, STRICT gt\_loader, positive no-person/audio/MD5/source-path scan) + the M2 refusal gate (raise unless owned AND minors\_free AND non-blank license). The golden-crossos CI job runs `--check` + tests/test\_golden\_fixtures.py on ubuntu/windows/macOS.  
ALREADY VERIFIED ? do NOT re-report as new findings: full suite 759 pass/7 skip; coverage 80.49%; ruff+mypy clean; cross-OS green on all 3 OSes for the COMMITTED fixtures; CONTROL avoids numpy \*execution\*.  
Find what that MISSED: real gaps / bugs / edge-cases / privacy holes / determinism holes / over-claims. Cite file:line. Do NOT invent issues ? if a lens is clean, say so honestly and set verdict accordingly.

### 2. assistant (2026-06-16T06:27:05.050Z)

I'll audit the Golden Regression Fixtures feature through the privacy/de-attribution lens. Let me start by reading the core files.

### 3. user (2026-06-16T06:27:06.348Z)

```
1      """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3      `docs/GOLDEN_REGRESSION_FIXTURES.md` ($4, $6, $7). This module is the **single shared
```

```

4      replay code path** used by both the freeze tool and the characterization gate, and it
5      deliberately stays on the **pure-stdlib CONTROL path** (D5 in that doc): the learned
6      variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7      across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8      only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9      merges overlaps, and uses **no numpy**.
10
11     The replay reuses the already-shipped eval stack VERBATIM::
12
13         parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
14         ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
15         ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
16
17     Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
18
19     * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
20     provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
21     * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"``.
22     * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
23     person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
24     * The windowing preset params ``(vt, mg, mwd, max_win)`` are **pinned inside each
fixture's
25     provenance** and passed explicitly to ``build_candidates`` ? we do NOT rely on the
named
26     ``windowing.SERVED`` constant, so a future SERVED retune cannot silently churn
fixtures.
27
28     §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
29     suite proves only that the deterministic post-trajectory transforms are unchanged for
the
30     frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
31     that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
32     field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
33
34     Pure + offline + stdlib. The replay imports `tracknet_runner` (which is itself stdlib-
only
35     for the parse/window math ? no TF), but pulls in no numpy/cv2 on the CONTROL path.
36     """
37     from __future__ import annotations
38
39     import argparse
40     import json
41     import math
42     import os
43     import re
44     import sys
45     from dataclasses import dataclass
46     from pathlib import Path
47     from typing import Any, Dict, List, Optional, Sequence, Tuple
48
49     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-----
50     from backend.eval import distill_local
51     from backend.eval.experiment import CONTROL, VideoCandidates, predict
52     from backend.eval.gt_loader import load_gt_intervals
53     from backend.eval.metrics import score
54     from backend.eval.regression import gt_hash

```

```

55     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
56
57     Interval = Tuple[float, float]
58
59     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
60     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
61     CURRENT_SCHEMA = 1
62
63     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
64     #: works
65     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
66     #: root.
67     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
68
69     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
70     #: merge_gap,
71     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
72     #: LITERAL
73     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
74     #: constant so a
75     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
76     #: fixture also
77     #: re-states these in its provenance; the replay reads them from there.
78     PINNED_VT = 0.02
79     PINNED_MERGE_GAP = 2.0
80     PINNED_MIN_WINDOW = 2.0
81     PINNED_MAX_WINDOW = 0.0 # 0 = bounded-windowing OFF (W1), matching the SERVED default
82
83     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
84     #: stated
85     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
86     #: exactly these.
87     SHUTTLE_FEATURES: Tuple[str, ...] = (
88         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
89     )
90     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
91         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
92     )
93
94     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
95     #: appear
96     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
97     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
98
99     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
100     _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
101
102     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
103     #: refuses to
104     #: emit a fixture whose written text contains one (it would be an attribution back-
105     #: channel, §4c/§5).
106     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
107
108     # =====
109     # Synthetic trajectory generation (deterministic ? NO random)
110     # =====
111
112     @dataclass(frozen=True)
113     class RallySpec:
114         """One in-flight rally segment: the shuttle oscillates across the net midline.
115
116         ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
117         period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the

```

```

109         net_midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
110         ""
111
112         start_frame: int
113         n_frames: int
114         period: float = 30.0
115         amplitude: float = 400.0
116
117
118     @dataclass(frozen=True)
119     class GapSpec:
120         """Dead-time between rallies ? what separates one window from the next.
121
122         ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
123         ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
124         end an active run so the next rally forms a distinct window.
125         ""
126
127         start_frame: int
128         n_frames: int
129         kind: str = "still" # "still" | "occluded"
130
131
132     @dataclass(frozen=True)
133     class FixtureSpec:
134         """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
135
136         ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
137         "labels.csv". For a synthetic fixture they line up with the rallies by construction,
138         which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
139         field-quality one (§6).
140         ""
141
142         slug: str
143         description: str
144         fps: float
145         frame_width: float
146         frame_height: float
147         segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
148         gts: Tuple[Interval, ...]
149         net_midline: float = 640.0
150
151
152     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
153                      ) -> List[Tuple[int, int, float, float]]:
154         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
155         rows: List[Tuple[int, int, float, float]] = []
156         y_mid = frame_height / 2.0
157         for i in range(spec.n_frames):
158             x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
159             # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
160             y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
161             rows.append((spec.start_frame + i, 1, x, y))
162         return rows
163
164
165     def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
166                   ) -> List[Tuple[int, int, float, float]]:
167         """Dead-time rows: resting-visible (still) or absent (occluded)."""
168         rows: List[Tuple[int, int, float, float]] = []
169         y_mid = frame_height / 2.0
170         for i in range(spec.n_frames):

```

```

171         if spec.kind == "occluded":
172             rows.append((spec.start_frame + i, 0, 0.0, 0.0))
173         elif spec.kind == "still":
174             rows.append((spec.start_frame + i, 1, net_midline, y_mid))
175         else: # pragma: no cover - guarded by builder
176             raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'/'occluded')")
177     return rows
178
179
180 def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
181     """Build the deterministic ``Frame, Visibility, X, Y`` rows for a synthetic
182     scenario."""
183     rows: List[Tuple[int, int, float, float]] = []
184     for seg in spec.segments:
185         if isinstance(seg, RallySpec):
186             rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
187         elif isinstance(seg, GapSpec):
188             rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
189         else: # pragma: no cover - guarded by dataclass shape
190             raise TypeError(f"unknown segment type {type(seg).__name__}")
191     rows.sort(key=lambda r: r[0])
192     return rows
193
194 def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
195 None:
196     """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp
197     floats."""
198     lines = ["Frame,Visibility,X,Y"]
199     for frame, vis, x, y in rows:
200         lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
201     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
202
203 def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
204     """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
205     LF."""
206     lines = ["start,end"]
207     for s, e in gts:
208         lines.append(f"{_fmt(s)},{_fmt(e)}")
209     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
210
211 # =====
212 # Built-in synthetic scenarios (the 3 committed slugs)
213 # =====
214
215 def _fr(seconds: float, fps: float) -> int:
216     return int(round(seconds * fps))
217
218
219 def builtin_specs() -> List[FixtureSpec]:
220     """The committed synthetic scenarios. Deterministic; chosen so each exercises a
221     distinct
222     windowing behaviour (one window / dead-time split / occlusion split)."""
223     fps = 30.0
224     fw, fh = 1280.0, 720.0
225     net = 640.0
226
227     # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
228     crossings>0.
229     single = FixtureSpec(
230         slug="clean_single_rally",
231         description="One continuous ~4s rally; shuttle oscillates across the net ? 1

```

```

window.",
230         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
231         segments=(RallySpec(start_frame=0, n_frames=120),),
232         # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
233         gts=((0.0, 4.0),),
234     )
235
236     # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
237     #     low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
238     r0 = RallySpec(start_frame=0, n_frames=120)
239     gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
240     r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
241     multi = FixtureSpec(
242         slug="multi_rally_with_deadtime",
243         description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
244         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
245         segments=(r0, gap_still, r1),
246         gts=((0.0, 4.0), (7.0, 11.0)),
247     )
248
249     # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
250     #     visibility break ? proves occlusion (not just low velocity) separates windows.
251     o0 = RallySpec(start_frame=0, n_frames=120)
252     gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
253     o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
254     occ = FixtureSpec(
255         slug="occlusion_break",
256         description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
257         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
258         segments=(o0, gap_occ, o1),
259         gts=((0.0, 4.0), (7.0, 11.0)),
260     )
261
262     return [single, multi, occ]
263
264
265     def _spec_by_slug() -> Dict[str, FixtureSpec]:
266         return {s.slug: s for s in builtin_specs()}
267
268
269     # =====
270     # The shared replay (CONTROL / pure-stdlib path)
271     # =====
272
273
274     def _fmt(x: float) -> str:
275         """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
276         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
277         if r == 0.0:
278             r = 0.0
279         return f"{r:.6f}"
280
281
282     def _round(x: float) -> float:
283         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
284         return 0.0 if r == 0.0 else r
285
286
287     def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
288         """The windowing proposal tuple, read from the fixture provenance when present (the

```

```

pinned
289         params travel with the fixture), else the module-level pinned defaults. NEVER
windowing.SERVED."""
290         if prov and "windowing" in prov:
291             w = prov["windowing"]
292             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
293         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
294
295
296     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
297         """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.
298
299         Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
300         ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
301
302         parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
303         cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
304
305         Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
306         control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
307         merges overlaps; NO numpy is touched.
308         """
309         fdir = _resolve_dir(slug_or_dir)
310         slug = fdir.name
311         traj_csv = fdir / "trajectory.csv"
312         labels_csv = fdir / "labels.csv"
313         prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
314
315         fps, fw = _fps_fw(prov, slug)
316         iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
317         proposal = _pinned_proposal(prov)
318
319         intervals, feature_rows = distill_local.build_candidates(
320             str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
321
322         # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
323         # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
324         # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
325         features: Dict[str, List[float]] = {
326             name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
327         }
328         gts = load_gt_intervals(str(labels_csv), strict=True)
329         vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
330
331         preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
332         sc = score(preds, gts, iou)
333
334         candidates: List[Dict[str, Any]] = []
335         for j, (s, e) in enumerate(intervals):
336             shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
337             # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
338             shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
339             candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})

```

```

340
341     return {
342         "slug": slug,
343         "schema_version": CURRENT_SCHEMA,
344         "iou": _round(iou),
345         "candidates": candidates,
346         "control_segments": [[_round(s), _round(e)] for s, e in preds],
347         "score": {
348             "iou_threshold": _round(sc.iou_threshold),
349             "num_pred": int(sc.num_pred),
350             "num_gt": int(sc.num_gt),
351             "true_positives": int(sc.true_positives),
352             "false_positives": int(sc.false_positives),
353             "false_negatives": int(sc.false_negatives),
354             "precision": _round(sc.precision),
355             "recall": _round(sc.recall),
356             "f1": _round(sc.f1),
357             "mean_iou": _round(sc.mean_iou),
358             "mean_start_error": _round(sc.mean_start_error),
359             "mean_end_error": _round(sc.mean_end_error),
360         },
361         "gt_hash": gt_hash(gts),
362     }
363
364
365     # =====
366     # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
367     # =====
368
369
370     def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
371                     ) -> Dict[str, Any]:
372         """The committed provenance block ? pins the windowing + privacy/licensing tags
373         (§4c, §5)."""
374         desc = ""
375         if isinstance(spec_or_dir, FixtureSpec):
376             desc = spec_or_dir.description
377         return {
378             "slug": slug,
379             "schema_version": CURRENT_SCHEMA,
380             "kind": "synthetic",
381             "license": "synthetic",
382             "minors_free": True,
383             "description": desc,
384             "fps": _round(fps),
385             "frame_width": _round(fw),
386             "iou": _round(iou),
387             "windowing": {
388                 "vt": PINNED_VT,
389                 "mg": PINNED_MERGE_GAP,
390                 "mwd": PINNED_MIN_WINDOW,
391                 "max_win": PINNED_MAX_WINDOW,
392             },
393             "paths": {
394                 "trajectory": "trajectory.csv",
395                 "labels": "labels.csv",
396                 "expected": "expected.json",
397             },
398             "note": (
399                 "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
400                 "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
401                 (§6).",
402             ),
403         }

```



```

403
404     def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
405                        iou: float = 0.5) -> Dict[str, Any]:
406         """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
COMMITTED
407         ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).
408
409         Idempotent: rewrites the same bytes given the same committed inputs. Requires the
trajectory
410         + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
411         """
412         fdir = fixtures_dir / slug
413         traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
414         if not traj_csv.is_file() or not labels_csv.is_file():
415             raise FileNotFoundError(
416                 f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
417
418         # Provenance first (it pins the windowing the replay reads), then replay ? expected.
419         spec = _spec_by_slug().get(slug)
420         fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
421         prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
422         _write_json(fdir / "provenance.json", prov)
423
424         expected = replay_fixture(fdir)
425         _write_json(fdir / "expected.json", expected)
426         return expected
427
428
429     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
430         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
431         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
432         specs = builtin_specs()
433         fixtures_dir.mkdir(parents=True, exist_ok=True)
434         slugs: List[str] = []
435         for spec in specs:
436             fdir = fixtures_dir / spec.slug
437             fdir.mkdir(parents=True, exist_ok=True)
438             rows = make_synthetic_trajectory(spec)
439             write_trajectory_csv(rows, fdir / "trajectory.csv")
440             write_labels_csv(spec.gts, fdir / "labels.csv")
441             freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
442             slugs.append(spec.slug)
443         _write_manifest(specs, fixtures_dir)
444         return slugs
445
446
447     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
448                        fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
449         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
450         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
regression; the
451         mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
452         if all_slugs:
453             targets = [d.name for d in sorted(fixtures_dir.iterdir())
454                        if d.is_dir() and (d / "trajectory.csv").is_file()]
455         elif slug:
456             targets = [slug]
457         else:
458             raise ValueError("refresh_snapshot needs --slug S or --all")

```

```

459         for s in targets:
460             freeze_fixture(s, fixtures_dir=fixtures_dir)
461         return targets
462
463
464     # =====
465     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
466     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
467     # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
468     # =====
469
470
471     class RefusalError(ValueError):
472         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
473
474         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
475         it) that
476         names *why* the unsafe public-fixture path was refused. The refusal is the code-
477         enforced
478         provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
479         minors-excluded
480         / license flags are human-asserted, but the gate makes the unsafe path impossible to
481         reach
482         *silently* ? a fixture cannot be written unless all three are affirmatively set.
483         """
484
485     def _new_surrogate_slug() -> str:
486         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).
487
488         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
489         random,
490         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
491         over a
492         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
493         off-git.
494         """
495         return f"fx_{os.urandom(8).hex()}"
496
497
498     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). Source-
499     path tokens
500     #: that collide with these are NOT a leak ? exclude them so a short source stem like
501     ``traj`` can't
502     #: false-positive against the literal
503     ``trajectory.csv``/``labels.csv``/``expected.json``.
504     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
505         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
506         "provenance", "provenance.json",
507     )
508
509
510     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
511         """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
512         attributable.
513
514         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
515         substring,
516         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
517         path
518         (full path / basename / stem) ? excluding the fixed schema filename literals the
519         writers
520         always emit. The trajectory is shuttle-only by nature; this guards against a
521         *regression*
522         re-introducing a person/audio/pose stream or an id/path back-channel ? never the

```

```

silent default.
509     """
510     low = text.lower()
511     for key in FORBIDDEN_KEYS:
512         if key in low:
513             raise RefusalError(
514                 f"{where}: forbidden key substring {key!r} found in written output ? "
515                 f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
516                 f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
517     m = _MD5_RE.search(text)
518     if m:
519         raise RefusalError(
520             f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
521             f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
522     for sp in source_paths:
523         if not sp:
524             continue
525         p = Path(sp)
526         for token in (sp, p.name, p.stem):
527             t = (token or "").strip().lower()
528             # Skip a token that is itself a substring of a schema-literal the writers
always emit
529             # (e.g. a source stem ``traj`` ? ``trajectory.csv``) ? that collision is not
a leak.
530             if not t or any(t in lit for lit in _SCHEMA_FILENAME_LITERALS):
531                 continue
532             if t in low:
533                 raise RefusalError(
534                     f"{where}: source-path token {token!r} found in written output ? the
source "
535                     f"filename/path must never be persisted (§4c metadata strip).")
536
537
538     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
539                               ) -> List[Tuple[int, int, float, float]]:
540         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.
541
542         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
543         pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
544         """
545         rows: List[Tuple[int, int, float, float]] = []
546         for p in points:
547             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
548         rows.sort(key=lambda r: r[0])
549         return rows
550
551
552     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
553         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant)."""
554         return [(s - t0_sec, e - t0_sec) for s, e in gts]
555
556
557     def freeze_real_fixture(
558         label: str,
559         trajectory_csv_path: Any,
560         labels_csv_path: Any,
561         *,
562         license: str,

```

```

563     minors_free: bool,
564     owned: bool,
565     fps: float,
566     frame_width: float,
567     fixtures_dir: Path = FIXTURES_DIR,
568     slug: Optional[str] = None,
569     time_origin_reset: bool = True,
570     source_note: str = "",
571 ) -> Dict[str, Any]:
572     """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
573     Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
row P1.)
574
575     The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
576     ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string.
577     The committed bytes carry NO filename / video_id / match / player / capture-date ?
only an
578     opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by
default), the
579     reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy
assertion
580     re-reads the written ``expected.json`` + ``provenance.json`` and refuses if anything
attributable
581     slipped in.
582
583     NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
584     only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
585     attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
586     owner-asserted Fork-A / minors / biometric-exclusion flags.
587
588     Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
589     """
590     # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)
-----
591     reasons: List[str] = []
592     if minors_free is not True:
593         reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
594             "of a minor is prohibited ? §3, guardrail 'exclude minors')")
595     if owned is not True:
596         reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
597             "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
598     if not (isinstance(license, str) and license.strip()):
599         reasons.append("license must be a non-blank string (per-record provenance tag ?
$4c "
600             "'provenance hard-fail gate')")
601     if reasons:
602         raise RefusalError(
603             "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
604             "cure provenance/licensing; this gate sits behind counsel sign-off,
$6/P2):\n - "
605             + "\n - ".join(reasons))
606
607     traj_path = Path(str(trajectory_csv_path))
608     labels_path = Path(str(labels_csv_path))
609

```

```

610         # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
611         out_slug = slug if slug is not None else _new_surrogate_slug()
612
613         # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
-----
614         points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
615         if not points:
616             raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
617         gts = load_gt_intervals(str(labels_path), strict=True)
618
619         # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, §4c)
-----
620         # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
because
621         # metrics.score is built from pure differences (interval_iou, |start-start|, |end-
end|), the
622         # score is bit-identical with vs without reset ? only the absolute timeline is
severed. The
623         # quality-floor / characterization snapshot is therefore unchanged (asserted in the
tests).
624         if time_origin_reset:
625             t0_frame = min(int(p.frame) for p in points)
626             t0_sec = t0_frame / float(fps)
627         else:
628             t0_frame, t0_sec = 0, 0.0
629         rows = _reset_trajectory_rows(points, t0_frame)
630         reset_gts = _reset_labels(gts, t0_sec)
631
632         # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture)
-----
633         fdir = fixtures_dir / out_slug
634         fdir.mkdir(parents=True, exist_ok=True)
635         write_trajectory_csv(rows, fdir / "trajectory.csv")
636         write_labels_csv(reset_gts, fdir / "labels.csv")
637
638         iou = 0.5
639         prov = {
640             "slug": out_slug,
641             "schema_version": CURRENT_SCHEMA,
642             "kind": "cleared_real",
643             "label": str(label),
644             "license": license,
645             "minors_free": True,
646             "owned": True,
647             "fps": _round(fps),
648             "frame_width": _round(frame_width),
649             "iou": _round(iou),
650             "time_origin_reset": bool(time_origin_reset),
651             "windowing": {
652                 "vt": PINNED_VT,
653                 "mg": PINNED_MERGE_GAP,
654                 "mwd": PINNED_MIN_WINDOW,
655                 "max_win": PINNED_MAX_WINDOW,
656             },
657             "paths": {
658                 "trajectory": "trajectory.csv",
659                 "labels": "labels.csv",
660                 "expected": "expected.json",
661             },
662             "gt_hash": gt_hash(reset_gts),
663             "source_note": str(source_note),
664             "note": (
665                 "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,

```

```

biometric-"
666         "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
667         "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
668         "Tier-0 / CONTROL-only. $6: behaviour-locking, not correctness; sits behind
counsel (P2)."
```

```

669     ),
670 }
671 _write_json(fdir / "provenance.json", prov)
672
673 expected = replay_fixture(fdir)          # REUSE M1 ? correct-by-construction
snapshot
674     _write_json(fdir / "expected.json", expected)
675
676     # --- (f) POSITIVE privacy assertion AFTER writing (scan the written text, $4c / ?)
-----
677     source_paths = [str(traj_path), str(labels_path)]
678     _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
679                     where=f"{out_slug}/expected.json", source_paths=source_paths)
680     _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
681                     where=f"{out_slug}/provenance.json", source_paths=source_paths)
682
683     # --- (g) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
684     _append_manifest_entry(prov, fixtures_dir)
685     return expected
686
687
688 def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
689     """A manifest row for a single fixture, derived from its provenance block."""
690     slug = prov["slug"]
691     return {
692         "slug": slug,
693         "fps": _round(float(prov["fps"])),
694         "frame_width": _round(float(prov["frame_width"])),
695         "windowing": dict(prov.get("windowing", {})),
696         "kind": prov.get("kind", "synthetic"),
697         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
698         "paths": {
699             "dir": slug,
700             "trajectory": f"{slug}/trajectory.csv",
701             "labels": f"{slug}/labels.csv",
702             "expected": f"{slug}/expected.json",
703             "provenance": f"{slug}/provenance.json",
704         },
705     }
706
707
708 def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
709     """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
710     entries = load_manifest(fixtures_dir)
711     entries = [e for e in entries if e.get("slug") != prov["slug"]]
712     entries.append(_manifest_entry_for(prov))
713     entries.sort(key=lambda e: str(e.get("slug")))
714     _write_json(fixtures_dir / "manifest.json", entries)
715
716
717     # =====
718     # Manifest + loaders
719     # =====
720
721
722 def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:

```

```

723     entries = [{
724         "slug": spec.slug,
725         "fps": _round(spec.fps),
726         "frame_width": _round(spec.frame_width),
727         "windowing": {
728             "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
729             "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
730         },
731         "kind": "synthetic",
732         "schema_version": CURRENT_SCHEMA,
733         "paths": {
734             "dir": spec.slug,
735             "trajectory": f"{spec.slug}/trajectory.csv",
736             "labels": f"{spec.slug}/labels.csv",
737             "expected": f"{spec.slug}/expected.json",
738             "provenance": f"{spec.slug}/provenance.json",
739         },
740     } for spec in specs]
741     _write_json(fixtures_dir / "manifest.json", entries)
742
743
744     def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
745         """Load the fixtures manifest (returns [] if absent)."""
746         path = fixtures_dir / "manifest.json"
747         if not path.is_file():
748             return []
749         data = _read_json(path)
750         if not isinstance(data, list): # pragma: no cover - corrupt manifest
751             raise ValueError(f"{path}: manifest must be a JSON array")
752         return data
753
754
755     def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
756         """Load one fixture's committed artifacts: `{provenance, expected, dir}`. Refuses
a fixture
757         whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible)."""
758         fdir = fixtures_dir / slug
759         prov = _read_json(fdir / "provenance.json")
760         expected = _read_json(fdir / "expected.json")
761         sv = int(expected.get("schema_version", prov.get("schema_version", 0)))
762         if sv > CURRENT_SCHEMA:
763             raise ValueError(
764                 f"{slug}: schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA} ? upgrade
the harness")
765         return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
766
767
768     # =====
769     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
770     # =====
771
772
773     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
774         """Return a list of human-readable mismatch messages (empty == match).
775
776         Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
777         TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
778         equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
779         safe (the repo runs with core.autocrlf=true)."""
780         out: List[str] = []
781

```

```

782     def exact(path: str, a: Any, b: Any) -> None:
783         if a != b:
784             out.append(f"{path}: {a!r} != {b!r} (exact)")
785
786     def approx(path: str, a: Any, b: Any) -> None:
787         try:
788             if abs(float(a) - float(b)) > _FLOAT_TOL:
789                 out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
790         except (TypeError, ValueError):
791             out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
792
793     exact("slug", recomputed.get("slug"), committed.get("slug"))
794     exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
795     exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
796     approx("iou", recomputed.get("iou"), committed.get("iou"))
797
798     rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
799     if len(rc) != len(cc):
800         out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
801     else:
802         for i, (r, c) in enumerate(zip(rc, cc)):
803             approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
804             approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
805             rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
806             if set(rs) != set(cs):
807                 out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
808             else:
809                 for k in rs:
810                     if k == "net_crossings":
811                         exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
812                     else:
813                         approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
814
815     rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
816     if len(rseg) != len(cseg):
817         out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
818     else:
819         for i, (r, c) in enumerate(zip(rseg, cseg)):
820             approx(f"control_segments[{i}][0]", r[0], c[0])
821             approx(f"control_segments[{i}][1]", r[1], c[1])
822
823     rsc, csc = recomputed.get("score", {}), committed.get("score", {})
824     int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
825     float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
826                     "mean_start_error", "mean_end_error")
827     for k in int_fields:
828         exact(f"score.{k}", rsc.get(k), csc.get(k))
829     for k in float_fields:
830         approx(f"score.{k}", rsc.get(k), csc.get(k))
831
832     return out
833
834
835     # =====
836     # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
837     # =====
838
839
840     def _write_json(path: Path, obj: Any) -> None:
841         text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
842         path.write_text(text + "\n", encoding="utf-8", newline="\n")

```



```

843
844
845 def _read_json(path: Path) -> Any:
846     with open(path, encoding="utf-8") as f:
847         return json.load(f)
848
849
850 def _resolve_dir(slug_or_dir: Any) -> Path:
851     if isinstance(slug_or_dir, Path):
852         return slug_or_dir
853     p = Path(str(slug_or_dir))
854     if p.is_dir():
855         return p
856     return FIXTURES_DIR / str(slug_or_dir)
857
858
859 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
860     if prov and "fps" in prov and "frame_width" in prov:
861         return float(prov["fps"]), float(prov["frame_width"])
862     spec = _spec_by_slug().get(slug)
863     if spec is not None:
864         return spec.fps, spec.frame_width
865     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
866
867
868 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
869     prov_path = fdir / "provenance.json"
870     prov = _read_json(prov_path) if prov_path.is_file() else None
871     return _fps_fw(prov, slug)
872
873
874 # =====
875 # CLI
876 # =====
877
878
879 def _cmd_check(fixtures_dir: Path) -> int:
880     manifest = load_manifest(fixtures_dir)
881     if not manifest:
882         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
883         return 1
884     failures = 0
885     for entry in manifest:
886         slug = entry["slug"]
887         committed = _read_json(fixtures_dir / slug / "expected.json")
888         recomputed = replay_fixture(fixtures_dir / slug)
889         mismatches = compare_expected(recomputed, committed)
890         if mismatches:
891             failures += 1
892             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
f"(characterization = behaviour-locking, NOT correctness ? $6):")
893             for m in mismatches:
894                 print(f"    - {m}")
895         else:
896             sc = recomputed["score"]
897             print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
898     print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
"
899
900     f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
901     return 0 if failures == 0 else 1
902
903
904

```

```

905     def main(argv: Optional[Sequence[str]] = None) -> int:
906         ap = argparse.ArgumentParser(
907             description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
908         g = ap.add_mutually_exclusive_group(required=True)
909         g.add_argument("--freeze-synthetic", action="store_true",
910             help="generate synthetic trajectory+labels for all built-in
scenarios, then "
911                 "freeze expected.json+provenance.json + write manifest.json")
912         g.add_argument("--refresh-snapshot", action="store_true",
913             help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
914         g.add_argument("--check", action="store_true",
915             help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
916         g.add_argument("--freeze-real", action="store_true",
917             help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
918                 "fixture; REFUSED without --owned --minors-free and a non-blank
--license")
919         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
920                 "--freeze-real (default: a random fx_<hex> id)")
921         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
922         ap.add_argument("--reason", help="recorded reason for --refresh-snapshot (review the
diff!)")
923         # --freeze-real (P1 / M2 refusal gate) options:
924         ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
925         ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
(start,end)")
926         ap.add_argument("--license", dest="license_", default=None,
927             help="--freeze-real) the source license tag (required, non-blank)")
928         ap.add_argument("--owned", action="store_true",
929             help="--freeze-real) assert owner-recorded Fork-A source
(required)")
930         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
931             help="--freeze-real) assert the source contains NO minors
(required)")
932         ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
933         ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
real) source frame width (px)")
934         ap.add_argument("--label", default="cleared_real", help="--freeze-real) non-
identifying scenario label")
935         ap.add_argument("--source-note", dest="source_note", default="",
936             help="--freeze-real) non-identifying note (NEVER a path/name/id)")
937         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
938             help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
939         ap.set_defaults(time_origin_reset=True)
940         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
941             help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
942         args = ap.parse_args(list(argv) if argv is not None else None)
943
944         fixtures_dir: Path = args.fixtures_dir
945
946         if args.freeze_synthetic:
947             slugs = freeze_synthetic(fixtures_dir)
948             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'
'.join(slugs)}")
949             print(f"    -> {fixtures_dir}")
950             return 0

```

```

951
952     if args.refresh_snapshot:
953         if not args.slug and not args.all:
954             ap.error("--refresh-snapshot requires --slug S or --all")
955         if not args.reason:
956             print("[golden-fixtures] WARNING: no --reason given. --refresh-snapshot can
957 "
958             "rubber-stamp a real regression ? review the visible diff ($6).",
959 file=sys.stderr)
960     targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
961 fixtures_dir=fixtures_dir)
962     print(f"[golden-fixtures] refreshed snapshot for: {' ', '.join(targets)}"
963         + (f" (reason: {args.reason})" if args.reason else ""))
964     return 0
965
966 if args.freeze_real:
967     missing = [n for n, v in (("--trajectory", args.trajectory),
968                             ("--labels", args.labels),
969                             ("--fps", args.fps),
970                             ("--frame-width", args.frame_width)) if v is None]
971     if missing:
972         ap.error("--freeze-real requires " + " ".join(missing))
973     try:
974         expected = freeze_real_fixture(
975             args.label, args.trajectory, args.labels,
976             license=args.license_ if args.license_ is not None else "",
977             minors_free=args.minors_free, owned=args.owned,
978             fps=args.fps, frame_width=args.frame_width,
979             fixtures_dir=fixtures_dir, slug=args.slug,
980             time_origin_reset=args.time_origin_reset, source_note=args.source_note)
981     except RefusalError as e:
982         # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
983         zero.
984         print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
985 file=sys.stderr)
986         return 2
987     print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
988 "
989         f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
990     print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
991 to the "
992         "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
993 (P2).")
994     return 0
995
996 # --check
997 return _cmd_check(fixtures_dir)
998
999 if __name__ == "__main__":
1000     raise SystemExit(main())
1001

```

#### 4. user (2026-06-16T06:27:07.079Z)

```

1      # Golden Regression Fixtures ? video-free, non-attributable regression suite
2
3      > **Status:** `IN PROGRESS` ? owner-gated . **Owner:** Avi . **Created:** 2026-06-16 .
4      **Last updated:** 2026-06-16 . **Legal:** owner **accepted the researched risk** 2026-06-16
5      (conditions $3 stand; this is the owner's informed acceptance, **not** a retained-counsel
6      opinion)
7      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
8      DONE, per archive policy)
9      > **Tracking anchors:** $7 progress tracker is the source of truth; indexed in
10     `docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).

```

```

6      > **Relation to existing docs:** EXTENDS (does not supersede)
`docs/GOLDEN_DATA_SHARING.md` (#175); reuses the eval stack documented in `docs/EVALUATION.md` /
`docs/EXPERIMENT_HARNESS.md`; quality floor ties to `docs/QUALITY_ITERATIONS.md`.
7      > **Honesty note:** §3 (Legal) is **researched general information, NOT legal advice** ?
it gates on retained counsel (§8). §2/§4/§7 mark which claims are **[verified]** against code vs
**[design]** proposals.
8
9      ---
10
11     ## 0. TL;DR
12
13     Let a contributor on **another machine with no video, GPU, WSL, or DB** run the rally-
segmentation regression suite from **committed structured files**, so refactors and non-quality
PRs get a red/green side-effect signal. Two layers, built once and grown by accretion:
14
15     1. **Mechanics** ? freeze the deterministic pipeline at the **post-detector trajectory
seam** and replay it through the **already-shipped** eval stack (`experiment.py` / `metrics.py` /
`windowing.py` / `regression.py`), with two auto-discovered gates: a
**characterization/snapshot** gate (catches any behaviour change) and a **quality-floor** gate
(catches quality drops).
16     2. **Privacy/legal filter (SEALED-FIXTURES)** ? only **de-attributed, minimized, owner-
source, biometric-free** numbers are ever committed; richer/sensitive streams stay key-gated or
out-of-repo. Non-attributability comes from **deletion-at-source**, not encryption.
17
18     **The convergence that makes this clean:** the **privacy** answer (publish shuttle-only,
drop person/pose streams) and the **determinism** answer (freeze only the pure-stdlib `CONTROL`
path, keep the numpy/BLAS learned scorer off the cross-machine gate) point at the **same minimal
public tier**. That coincidence is the spine of the design.
19
20     **Two findings already actionable, independent of the rest:**
21     - ? **Live de-attribution bug** ** (verified): real player/venue names are committed
today in `eval_baselines/heuristic_lovo_n6.json`,
`eval_baselines/gemini_wrapper_2026-06-10.json`, and `backend/eval/eval_partial_labels.py` (the
last also has a personal `C:/Users/avidu/...` path) ? and in git history. ? **Deliverable P0.**
22     - ? **Determinism trap** ** (verified, corroborated by two independent red-teams): the
learned variants (`shuttle_only`, `fusion`) train a numpy/OpenBLAS `DYNAMIC_ARCH` logistic
regression whose decisions flip near the 0.5 threshold across CPUs ? would false-RED on a
contributor's machine. ? the cross-machine gate must be **CONTROL/static-only**.
23
24     ---
25
26     ## 1. Problem & goal
27
28     The rally-quality measurement is split across two machines by capability
(`docs/GOLDEN_DATA_SHARING.md`):
29     - `backend/eval/fusion_golden.py` **extracts** `_golden_features.json` ? needs the
**video + trajectory CSV + labels + GPU/WSL** (GPU box only).
30     - `backend/eval/fusion_compare.py` / `backend/eval/ablation.py` **re-score** those JSONs
on **pure CPU, no video** ? any machine.
31
32     The CPU re-scoring is **already** video-free, but the inputs live in gitignored `output/`
/ private OneDrive, so the regression tests that depend on them **skip everywhere**
(`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`). The
committed `eval_baselines/` baselines exist but `regression.py` still scores DB-stored
predictions (needs a pipeline run on the video).
33
34     **Goal:** commit a tiny, version-tagged, **numeric-only, non-attributable** per-fixture
corpus + auto-discovered gates so a clean `git clone` / CI gets a real red/green regression
signal with **zero video/GPU/DB**, while never leaking attributable or sensitive data.
35
36     ---
37
38     ## 2. Decisions locked
39
40     | # | Decision | Source | Implication |

```

```

41      |---|-----|-----|-----|
42      | D1 | **Corpus is owner-recorded / owned** | owner answer 2026-06-16 | Legal analysis
takes **Fork A** (clean): no third-party copyright/ToS infringement; concern is trade-
secret/moat + privacy. Strongest non-attributability (no public reference to correlate against).
|
43      | D2 | **Repo private now, may open-source later** | owner answer 2026-06-16 | Design
for **public-grade hygiene from day one** ? git history is permanent; nothing scrubable-later. |
44      | D3 | **Adults, consent not formalized** | owner answer 2026-06-16 | **Person/pose/gait
streams stay OUT of the public tier** (DPDP/GDPR derived-personal-data risk without consent).
Public tier = **shuttle + abstract intervals + non-biometric features only**. |
45      | D4 | **Freeze seam = post-detector trajectory** *(design)* | workflow-1 synthesis |
Widest deterministic surface; reuse the shipped eval stack verbatim; the raw per-frame CSV is
**never committed** (only ~5 aggregate, fps/res-invariant shuttle scalars/window). |
46      | D5 | **Cross-machine gate is CONTROL/static-only** *(design)* | both red-teams
[verified cause] | Learned variants (numpy/BLAS) are **owner-box/Tier-1 only**, asserted with
tolerance, never as a cross-machine byte-snapshot. |
47      | D6 | **Non-attributability = deletion-at-source, encryption = public-only barrier**
*(design)* | workflow-2 synthesis | A contributor who runs the tests necessarily reads
plaintext+key (DRM/analog-hole). So we **delete** attribution, not hide it. Encryption only
stops the no-key public. |
48      | D7 | **Two tiers** *(design)* | workflow-2 synthesis | **Tier-0** public/no-key
(shuttle-only, de-attributed). **Tier-1** key-gated/out-of-repo (person/audio, full corpus),
**skips cleanly when absent**. |
49
50      ---
51
52      ## 3. Legal foundation (researched ? gates on counsel, §8)
53
54      **Core verdict (settled across IN/US/EU):** the derived numbers ? per-frame shuttle
(x,y), rally start/end intervals, audio/court/aggregate-person feature values ? **do NOT inherit
the source video's copyright.** They are uncopyrightable *facts/measurements*, not authored
expression, and not a §101 "derivative work."
55
56      | Jurisdiction | Copyright of the data | Database right | Contract / ToS | Net |
57      |---|---|---|---|---|
58      | **India** (primary) | Not copyrightable as facts (*EBC v. D.B. Modak*); but
compilations get thin protection on a low "skill/labour" bar (*Burlington*, *OLX v. Padawan*) ?
only bites if **lifted from a 3rd-party dataset**, not self-derived | **None** (no sui-generis
right) ? a real permissiveness edge | Anti-scraping ToS (untested in court) + **IT Act
s.43/s.66** civil/criminal exposure for 3rd-party ingestion | **Owned-source + minors-excluded +
no person/pose = clean** |
59      | **US** | Not copyrightable (*Feist*; §102(b); *NBA v. Motorola*) | None (*Feist*
killed sweat-of-the-brow) | **Dominant risk.** ToS enforceable where copyright/CFAA fail
(*ProCD*, *hiQ* ? injunction + **forced deletion**). Remedy for redistribution is takedown, not
nominal damages | Publishable as facts **iff** owned/permissioned source + no barring ToS + no
person/biometric/minor stream |
60      | **EU / UK** | Not copyrightable (*Football Dataco v. Yahoo*) | **Genuinely
contestable.** *Football Dataco v. Sportradar*: recording pre-existing on-court facts =
**obtaining**, not "creating" ? our "we created it" premise is **overconfident**; harm test =
*CV-Online Latvia* | Ryanair v. PR Aviation: ToS can ban extraction/redistribution even with no
IP right | Get EU+UK advice before redistributing numeric streams at scale |
61
62      **Provenance is the decisive fork.** **Fork A (owner-recorded, unpublished)** ? the only
path safe to commit publicly; collapses to a trade-secret *choice* (Indian breach-of-confidence
/ Contract Act 1872 / IT Act s.72 ? **not** US DTSA) + privacy. **Fork B
(broadcast/YouTube/scraped)** ? do **not** commit; the extraction act needs a license or
now-**contested** US fair use (*Thomson Reuters v. Ross* 2025; USCO May-2025), and unlawful
acquisition independently sinks it (*Bartz v. Anthropic*, ~$1.5B settlement). Caveat: owner
footage **shot at a third party's event** (tournament/club) carries
ticket/venue/organiser/broadcaster terms ? treat as Fork B until cleared.
63
64      **Publish-by-stream (privacy):**
65      - ? **Publish-OK:** shuttle (x,y)+Visibility (inanimate object), abstract rally/GT
**intervals** (identity-stripped), non-biometric audio/court/aggregate-person features detached
from identity.

```

66 - ?? **\*\*Conditional (effectively never in v1):\*\*** person bounding boxes that **\*persistently**  
single out a player\* ? flips to GDPR Art.9 the moment it identifies. This architecture is prone  
to that flip ? treat per-player tracking as a tripwire.

67 - ? **\*\*Never publish:\*\*** pose/skeleton/**\*\*gait\*\*** (re-identifies at 80?95% even with faces  
blurred ? **\*Weiss et al. 2025\***), face crops, any named-player?movement mapping. Matches the  
existing "gait/biometrics strictly future (GDPR Art.9)" guardrail.

68 - ? **\*\*Minors flip everything:\*\*** DPDP "child" = under 18; s.9(3) **\*\*prohibits behavioural**  
**monitoring\*\*** (the Rule-12 exemptions don't cover this project). A persistent per-player  
signature on a minor is, conservatively, prohibited. **\*\*Exclude minors\*\*** from any persisted  
stream beyond inanimate shuttle/interval data.

69

70 **\*\*Non-attributability is relative, not absolute.\*\*** A per-video trajectory is a  
fingerprint (de Montjoye: 4 points ? 95% re-ID). It's only non-attributable because the **\*\*Fork-A**  
source stays unpublished\*\* (no public reference to correlate against). Per **\*EDPS v. SRB\*** (CJEU  
C-413/23 P, 2025) it **\*\*remains personal data to the holder\*\*** who can re-run the detector. So:  
conditional, revocable (publish the source and the link snaps back), forward-looking (re-assess  
as the public-video environment grows). Never market it as "anonymous."

71

72 **\*Key authorities:** Feist 499 U.S. 340; NBA v. Motorola 105 F.3d 841; 17 U.S.C.  
§101/§102(b); Sega v. Accolade 977 F.2d 1510; Football Dataco v. Yahoo (C-604/10) & v.  
Sportradar (C-173/11); BHB v. William Hill (C-203/02); CV-Online Latvia (C-762/19); Ryanair v.  
PR Aviation (C-30/14); Thomson Reuters v. Ross (D. Del. 2025); EBC v. D.B. Modak; DPDP Act 2023  
s.3/s.9 + Rules 2025; EDPS v. SRB (C-413/23 P). Full URLs in the research artifact (§10).

73

74 ---

75

76 **## 4. Architecture (design)**

77

78 **### 4a. Freeze seam + reuse map**

79 Freeze at the **\*\*trajectory CSV ? pure-Python replay\*\***. One shared helper  
`backend/eval/golden\_fixtures.py::replay\_fixture()` is the **\*single\*** code path used by both the  
freeze tool and both gates:

80 `parse\_trajectory\_csv ? windowing.build\_windows(preset) ? distill\_local.build\_candidates  
? rally\_gate net\_crossings ? experiment.predict(CONTROL) ? merge\_overlaps ? metrics.score`.

81 Reuses **\*\*verbatim\*\***: `metrics.score/interval\_iou`, `segmentation\_metrics`,  
`experiment.predict/compare/Variant.spec\_hash`, `windowing.WindowingPreset` (default SERVED),  
`regression.gt\_hash/save\_baseline + incommensurability guards`, `eval\_stats` (seed=0),  
`eval\_baselines/` committed-baseline precedent.

82

83 **### 4b. Tiered model (SEALED-FIXTURES)**

84 | Tier | Where | Key? | Contents | Test behaviour |

85 |---|---|---|---|---|

86 | **\*\*Tier-0\*\*** | `eval\_baselines/fixtures/` (tracked) | none | opaque `fixture\_id`,  
fps/frame\_width (quantized), relative `start`/`end`, **\*\*5 shuttle scalars\*\***, label, relative  
`gts`. **\*\*No person/audio/pose.\*\*** | **\*\*Unconditional\*\*** ? runs on every PR/fork; turns today's  
skipped litmus into a real public gate |

87 | **\*\*Tier-1\*\*** | OneDrive (default) **\*\*or\*\*** SOPS-encrypted in-repo | token/age key | full  
corpus **\*\*with\*\*** person/audio; authoritative ?F1/CI/sign-test + exact Gate-A litmus | **\*\*Skips**  
**cleanly\*\*** when key/data absent (generalize `\_golden\_present()` ? `\_tier1\_present()`) |

88

89 A fresh no-key clone is always **\*\*green\*\***; coverage never depends on a secret.

90

91 **### 4c. Anti-attribution measures (Tier-0)**

92 - **\*\*Opaque RANDOM surrogate id\*\*** (`os.urandom` + a private surrogate?source map kept  
off-git) ? **\*not\*** the MD5 `video\_id`, and **\*not\*** HMAC(salt, video\_id?name) (red-team: brute-  
forceable over a tiny known roster if the salt leaks). Strip the `name` field (today = source  
filename).

93 - **\*\*Metadata strip\*\*** ? no filename, video\_id, match/event/player identity, capture date.

94 - **\*\*Time-origin reset\*\*** ? subtract per-fixture `t0` from all start/end/gts. **\*\*Provably**  
**metric-invariant\*\*** (`metrics.interval\_iou` and start/end-error are pure differences) ? bit-  
identical scores, severed absolute timeline.

95 - **\*\*Stream minimization\*\*** ? Tier-0 keeps shuttle block only; person/audio excluded;  
pose/gait/face have **\*\*no field in the schema by design\*\***.

96 - **\*\*Raw-trajectory exclusion\*\*** ? never commit the per-frame `(Frame,Visibility,X,Y)`  
CSV.

```

97     - **Quantize fps/frame_width** ? red-team: at n=6 the raw values are a camera/venue
quasi-identifier.
98     - **Provenance hard-fail gate** ? the de-attribution pass refuses to emit a public
fixture for any record not tagged owner-recorded(Fork-A) + minors-excluded + biometric-excluded.
99
100    ### 4d. The two gates
101    - **Characterization/snapshot** ? replay fixture, **parse-compare** JSON (never byte-
compare ? CRLF/float-format safe; `core.autocrlf=true` here), exact on int/structural fields,
`approx(abs=1e-6)` on floats; stamp + check windowing-preset/label/gt fingerprints
(incommensurable ? loud fail). Catches any behaviour change even at constant F1. **CONTROL path
only** across machines.
102    - **Quality-floor** ? score vs committed labels; assert per-video + mean F1 ? floor in
`eval_baselines/regression_baseline.json` (created here; carries a **windowing fingerprint** so
a served-default change is flagged incommensurable, not silently compared); emit `eval_stats`
paired-delta-CI + sign test (NUMBERS INFORM); add a **metrics-vs-base-branch** paired delta.
Synthetic-tier floors measure **correctness, not field quality** (documented).
103
104    ### 4e. Serialization & optional protobuf
105    JSON with `sort_keys=True` + pre-quantized floats + LF is sufficient. **Protobuf is
optional (01)** ? adopt only if cross-OS byte-stable snapshots are wanted (proto3
`rally.fixture.v1`, codegen + `protoc && git diff --exit-code` guard, `*.pb binary -text`).
Protobuf is **encoding, not **secrecy** (trivially `--decode_raw`).
106
107    ### 4f. Encryption & keys (Tier-1 only)
108    **Default = out-of-repo + token** (existing OneDrive `RALLY_GOLDEN_DIR` seam ? zero
sensitive bytes in git history, sidesteps the legal "redistribution" act). **Alternative = SOPS
+ age** (per-recipient, revocable, value-level diffs) for Fork-A-clean-but-private data wanting
in-repo versioning. Rejected: git-crypt (whole-repo re-key), git-lfs (storage ? secrecy). CI
secret exposed **only to the Tier-1 job on trusted branches, never to fork-PR runners**. The
**HMAC/surrogate salt** (if used) and the surrogate?source map live with the private corpus,
rotated per release. Document explicitly that Tier-1 crypto defends **only** the no-key public.
109
110    ---
111
112    ## 5. Threat model (who's protected vs not)
113
114    | Actor | Protected against | NOT protected (by design / residual) |
115    |---|---|---|
116    | **TM1 no-key public** | map fixture?source/match/player; decrypt Tier-1; confirm a
source even holding the candidate file | read Tier-0 numbers + run shuttle-only harness
(intended; useless-in-isolation) |
117    | **TM2 keyed contributor** | recover filename/video_id/match/face/pose/gait ? *never
written* (survives the analog hole) | read every value the suite reads (unpreventable; not
pretended otherwise) |
118    | **TM3 CI runner** | leaked log can't leak attribution it never had | runs the suite
(intended); Tier-1 secret never on fork runners |
119    | **TM4 malicious keyed insider** | correlation needs a public reference; Fork-A source
is unpublished ? "not reasonably likely" | **residual:** conditional + revocable ? publish/leak
the source later and the link can snap back (forward-looking re-test duty) |
120    | **TM5 re-ID adversary** | source unpublished + only ~5 invariant scalars/window +
reset timeline ? no back-match | becomes live if a Fork-A source is ever published |
121    | **TM6 accidental plaintext commit** | required CI leak-scan rejects MD5/known-
name/non-surrogate/plaintext-SOPS | if the required check is disabled, a leak lands |
122
123    ---
124
125    ## 6. Honest limits ? what a green suite does NOT prove
126
127    - Green proves only that the **deterministic post-trajectory transforms** are unchanged
for the **frozen cases**. It does **not** cover: the WASB/TrackNet detector,
`_probe_fps_width`/cv2 frame-peek, the AI/provider handoff, chunking/re-encode, the ffmpeg
stitch, DB persistence/migrations, or the API filter. *A refactor that breaks decoding or the
detector passes these gates green.* The owner-box `regression.py` DB run remains the only full-
chain check.
128    - Characterization = **behaviour-locking, not **correctness** ? it freezes current

```

behaviour \*including current bugs\*. "Passing" = "behaviour unchanged", never "behaviour correct". This caveat must ride in the **\*\*test pass/fail message\*\***, not only the docs.

129 - **\*\*Synthetic-tier floors = plumbing correctness, NOT field quality.\*\*** Never cite synthetic F1 in `QUALITY\_ITERATIONS.md` / the paper as rally-detection performance (tag `kind: synthetic`).

130 - Tier-0 public coverage is **\*\*shuttle-only\*\***; the person/audio fusion ?F1 and exact Gate-A values need Tier-1 (skipped on no-key clones).

131 - Data is **\*\*not "anonymous" absolutely (EDPS v. SRB) ? relative, conditional, revocable, forward-looking.**

132 - Encryption gives **\*\*no secrecy from a running contributor\*\***; protobuf gives **\*\*no secrecy at all\*\***.

133 - De-attribution does **\*\*not\*\*** cure provenance/licensing ? it sits *\*behind\** the per-record provenance gate + counsel sign-off.

134 - **\*\*`--refresh-snapshot` can rubber-stamp a real regression; mitigation is **\*\*review of the visible diff\*\*** + a separate, deliberate floor-baseline update. Require a recorded reason.**

135

136 ---

137

138 **## 7. Deliverables & progress tracker ? **\*\*source of truth\*\*****

139

140 Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. One **\*\*small PR per row\*\***, branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.

141

142 | ID | Deliverable | Depends on | Counsel-gated? | Status | PR |

143 |----|-----|-----|-----|-----|----|

144 | **\*\*P0\*\*** | **\*\*Leak remediation + required CI leak-scan.\*\*** Scrub real names/paths in `eval\_baselines/heuristic\_lovo\_n6.json`, `gemini\_wrapper\_2026-06-10.json`, `backend/eval/eval\_partial\_labels.py`, `tests/test\_cleanup\_caches.py` ? opaque surrogate keys + private map; add CI leak-scan (MD5 / known-name / non-surrogate / plaintext-SOPS) as a **\*\*required status check\*\***; decide on git history (PR #77, accept vs `filter-repo`). | ? | No (owned data hygiene) | ? | ? |

145 | **\*\*M1\*\*** | **\*\*Fixture framework + synthetic Tier-0 + shared replay helper\*\*** (also delivered M2-core + M3 ? see below). `backend/eval/golden\_fixtures.py` (`replay\_fixture`, schema dispatch, `Path(\_\_file\_\_)`-relative) + 3 **\*\*tool-generated\*\*** synthetic fixtures + manifest + README + **\*\*`.gitattributes`** LF pin + `tests/test\_golden\_fixtures.py`. CONTROL-only / pure-stdlib; cross-OS verified. | ? | No (synthetic) | ? | [\[189\]\(https://github.com/avidullu/badminton-highlight-indexer/pull/189\)](https://github.com/avidullu/badminton-highlight-indexer/pull/189) |

146 | **\*\*M2\*\*** | **\*\*Freeze/refresh extractor.\*\*** Core (`freeze\_synthetic` / `refresh\_snapshot` / **\*\*`--check` CLI + idempotence test**) shipped in M1 (#189); the **\*\*`--license`/`--minors-free`/`--owned` **\*\*refusal gate\*\***** shipped with P1 (this PR). | M1 | No | ? | [\[189\]\(https://github.com/avidullu/badminton-highlight-indexer/pull/189\)](https://github.com/avidullu/badminton-highlight-indexer/pull/189) · **\*\*\_this PR\_** |

147 | **\*\*M3\*\*** | **\*\*Characterization gate\*\*** ? **\*\*`tests/test\_golden\_fixtures.py`**: parse-compared (exact-int / approx-float 1e-6), **\*\*CONTROL-only\*\***, positive no-person/audio assertion, perturbation + idempotence. **\*\*Shipped in M1 (#189).\*\*** (Windowing/label fingerprint guard + row-count cap ? fold into M4.) | M1 | No | ? | [\[189\]\(https://github.com/avidullu/badminton-highlight-indexer/pull/189\)](https://github.com/avidullu/badminton-highlight-indexer/pull/189) |

148 | **\*\*MX\*\*** | **\*\*Cross-OS determinism guard (CI).\*\*** Lightweight `golden-crossos` CI job (ubuntu/windows/macOS) running **\*\*`--check`** + the fixtures test ? locks the cross-OS guarantee (CONTROL replay is numpy-only, no GPU stack; a *\*separate fast job\**, not full-suite×3). | M1 | No | ? | [\[191\]\(https://github.com/avidullu/badminton-highlight-indexer/pull/191\)](https://github.com/avidullu/badminton-highlight-indexer/pull/191) |

149 | **\*\*M4\*\*** | **\*\*Quality-floor gate + first `regression\_baseline.json`.** **\*\*`tests/test\_golden\_quality\_floor.py`**; create **\*\*`eval\_baselines/regression\_baseline.json`** with **\*\*`windowing fingerprint`**; metrics-vs-base paired-delta; corpus-level (not per-slug) LOVO for runtime. | M1, M2 | No | ? | ? |

150 | **\*\*P1\*\*** | **\*\*De-attribution + minimization\*\*** ? **\*\*`golden\_fixtures.freeze\_real\_fixture()`** (extends the M1 module, not a separate script): **\*\*opaque random `fx\_hex` surrogate slug\*\***, metadata strip, metric-invariant **\*\*time-origin reset\*\***, strict GT loader, **\*\*positive no-person/audio/MD5/source-path assertion\*\***, manifest non-clobber; **\*\*`--freeze-real` CLI with the **\*\*refusal gate\*\*** (exit 2 unless owned + minors-free + license). Tooling-only, 12 synthetic tests; **\*\*no real data committed\*\***. | M1 | No (tooling) | ? | **\*\*\_this PR\_** |**

151 | **\*\*P2\*\*** | **\*\*Owner-accepted (researched-risk), 2026-06-16.\*\*** Owner accepted the researched IP/privacy analysis (§3) to proceed **\*\*under its conditions\*\*** (owned/Fork-A source · minors excluded · biometric/person streams excluded · de-attributed). Owner's informed risk acceptance ? **\*\*NOT a retained-counsel opinion\*\***; optional future counsel confirmation per §8. | P1 | Owner-accepted | ? | ? |



152 | **\*\*P3\*\*** | **\*\*Tier-0 REAL subset.\*\*** Commit de-attributed, minors/biometric-excluded, owned Fork-A real fixtures (unblocked by P2). Hard-gated **\*\*in code\*\*** by P1's refusal gate (owned + minors-free + license required) ? needs the owner's per-video clearance list. | P1, P2 | conditions §3 | ? | ? |

153 | **\*\*O1\*\*** | **\*(optional)\*** **\*\*Protobuf + deterministic serialization.\*\*** `rally.fixture.v1` proto3, codegen + `protoc`/`git diff --exit-code` guard, canonical writer, `.gitattributes` binary. Only if cross-OS byte-stable snapshots are wanted. | M1 | No | ? | ? |

154 | **\*\*O2\*\*** | **\*(optional)\*** **\*\*Tier-1 key-gated path.\*\*** `scripts/fetch\_golden.py` (OneDrive token, default) and/or SOPS+age; `\_tier1\_present()`; `RALLY\_GOLDEN\_DIR` wired; Tier-1 CI job gated on secret, never on fork runners. | M1 | No | ? | ? |

155 | **\*\*DOC\*\*** | **\*\*Finalize this doc + cross-links.\*\*** Add `eval\_baselines/fixtures/` row to `CODE\_MAP §0` + TEST MAP; cross-link `EVALUATION.md` / `EXPERIMENT\_HARNESS.md` / `GOLDEN\_DATA\_SHARING.md` / `QUALITY\_ITERATIONS.md`; update `docs/README.md` index + memory `current-state`/handoff. | M1..M4 | No | ? | ? |

156

157 **\*\*Must-fix-before-any-public-commit (red-team), folded into the rows above:\*\*** ? numeric-tolerance not byte-equality + multi-OS matrix + CONTROL-only (M3/D5/MX ?); ? scrub names in **\*\*all\*\*** files + history decision (P0); ? leak-scan a **\*\*required\*\*** check, hooks are convenience only (P0); ? person-optional loader + **\*\*positive\*\*** no-person assertion, don't trust the silent `0.0` default (P1 ?); ? random surrogate id not HMAC-over-roster (P1 ?); ? quantize/drop fps/frame\_width (P1 ? fps/fw retained as scale constants; quantize is a P3 option); ? document Tier-0 omits the fusion path (DOC); ? counsel + Fork-A + minors/biometric exclusion as hard preconditions (P2 owner-accepted; enforced in code by P1's refusal gate ?).

158

159 **### Progress log (quantitative ? coverage must hold/improve, CI incl. the 3-OS cross-OS job green)**

160 | PR | Deliverable(s) | Suite | Coverage | Notes |

161 |----|-----|-----|-----|-----|

162 | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) | M1 + M2-core + M3 | 746 pass / 7 skip | 80.41% | synthetic Tier-0 + CONTROL replay + characterization; cross-OS proven (CI Linux replayed Windows-frozen fixtures) |

163 | [#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191) | MX + tracker/legal reconcile | full suite green | 80.41% | `golden-crossos` CI job green on ubuntu+windows+macos ? cross-OS **\*\*enforced\*\***; legal ? owner-accepted |

164 | \_this PR\_ | P1 + M2 refusal gate | 759 pass / 7 skip | **\*\*80.49%\*\*** (+0.08) | `freeze\_real\_fixture` + refusal gate; **\*\*+13 tests\*\***; tooling-only, no real data committed |

165

166 ---

167

168 **## 8. Open owner / counsel questions**

169

170 **\*\*Counsel questions ? documented basis (the owner accepted the researched risk on 2026-06-16; see P2). These remain the record, and a retained-counsel confirmation is optional/recommended before scaling beyond a small owned, minors-free, de-attributed subset:\*\***

171 1. Does the EU/UK **\*\*sui-generis database right\*\*** subsist in our self-derived shuttle/rally streams on the *\*Sportradar\** "obtained vs created" reasoning, and does redistributing a de-attributed Tier-0 subset cause *\*CV-Online\** harm?

172 2. Post *\*Thomson Reuters v. Ross\** + USCO-2025, is **\*\*US** fair use for the extraction step\*\* defensible if the dataset could compete in a "data/analytics" market ? does publishing only numbers (never footage) matter?

173 3. India: are anti-scraping ToS enforceable; **\*\*IT Act s.43/s.66\*\*** exposure for any 3rd-party ingestion; does pending *\*ANI v. OpenAI\** (~Apr 2026) move it? Confirm **\*\*DPDP s.9/Rule 10\*\*** and whether per-player tracking is prohibited s.9(3) monitoring of a minor.

174 4. Sign-off on the **\*\*final committed Tier-0 subset + provenance log\*\***; is non-attributability-via-private-source a defensible posture to assert given its conditional/revocable nature?

175

176 **\*\*Owner (strategic):\*\***

177 5. Publishing the Fork-A subset **\*\*forfeits the trade-secret moat\*\*** (Indian breach-of-confidence framing) ? weighed against community/benchmark/paper aims. Your call.

178 6. **\*\*Forward-looking commitment:\*\*** do not publish a highlight reel of the *\*same\** source whose fixtures are committed (without re-clearing), and accept a periodic re-assessment as the public-video environment grows.

179 7. **\*\*Scope of first public commit:\*\*** synthetic-only (zero provenance risk, recommended) vs a counsel-cleared real subset.

```

180      8. **Tier-1 mechanism:** out-of-repo+token (recommended) vs SOPS+age in-repo. And: is
protobuf (01) wanted, or is canonical-JSON enough?
181
182      ---
183
184      ## 9. Definition of done
185
186      The project is **DONE** (flip `Status ? DONE`, archive) when:
187      - [ ] **P0, M1?M4, P1, DOC** are ? and merged.
188      - [ ] A clean `git clone` on **win + ubuntu + mac** runs `python run_all_tests.py` and
the **Tier-0 gates pass green with no key / no video / no GPU / no DB**.
189      - [ ] The **CI leak-scan is a required status check** and passes (no MD5 / real-name /
non-surrogate / plaintext-SOPS in the tree).
190      - [ ] Characterization + quality-floor gates are **parse-compared, CONTROL-only,
windowing-fingerprinted**, and demonstrably catch a seeded behaviour change.
191      - [ ] `docs/CODE_MAP.md $0` + TEST MAP updated; this doc cross-linked; `NEXT_STEPS.md` +
memory `current-state` point here.
192      - [ ] **Either** the public commit is synthetic-only, **or** (**P2 owner-accepted ? +
P3**) for a real-data commit, under the $3 conditions enforced **in code** by P1's refusal gate.
193      - [ ] $6 honest-limits caveat rides in the test pass/fail messages, not only the docs.
194
195      *Optional 01/02 may remain ? without blocking DONE (they're earned, not assumed).*
196
197      ---
198
199      ## 10. References
200
201      **Internal code anchors (verified):** `backend/eval/{regression.py,fusion_golden.py,fusi
on_compare.py,ablation.py,experiment.py,metrics.py>windowing.py,distill_local.py,gt_loader.py,ev
al_stats.py}` · `backend/pipeline/detectors/{base.py,tracknet_runner.py,rally_gate.py}` ·
`backend/utlils/hashing.py::compute_video_id` · `eval_baselines/{heuristic_lovo_n6.json,gemini_wr
apper_2026-06-10.json,experiment_leaderboard.json}` ·
`tests/test_ablation.py::test_litmus_reproduces_gate_a` · `.github/workflows/ci.yml`.
202      **Internal docs:** `GOLDEN_DATA_SHARING.md` (#175), `EVALUATION.md`,
`EXPERIMENT_HARNESS.md`, `QUALITY_ITERATIONS.md`, `MULTI_SIGNAL_FUSION_PLAN.md`, `CODE_MAP.md
$0`.
203      **Legal authorities:** see $3 (full URLs in the research workflow artifact). *This
section is researched general information, not legal advice.*
204
205      ---
206
207      ### Changelog
208      - **2026-06-16** ? Initial draft synthesized from two multi-agent workflows (mechanics
design + 2 critiques; legal memo IN/US/EU + SEALED-FIXTURES architecture + red-team). Verified
the live name-leak and the BLAS-determinism cause against code. Status: DRAFT, owner-gated.
209

```

## 5. assistant (2026-06-16T06:27:09.970Z)

Now let me read the supporting files to understand the full picture.

## 6. user (2026-06-16T06:27:10.931Z)

```

1      """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3      These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
an OWNED,
4      minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
fixture) and
5      its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
6
7      Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
NO GPU.
8      We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory``

```

```

and the
9     ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"
CSVs, then
10    prove:
11        * the refusal gate blocks every unsafe combination (M2);
12        * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no
FORBIDDEN_KEYS);
13        * the time-origin reset is metric-INVARIANT (score identical with/without; only
absolute
14            ``control_segments`` shift toward 0);
15        * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
16
17    $6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
18    are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
19    """
20    import pytest
21
22    from backend.eval import golden_fixtures as gf
23
24
25    # -----
26    # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
27    # -----
28
29
30    def _write_source(tmp_path, *, frame_offset: int = 0):
31        """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.
32
33        Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
34        ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
absolute
35        capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
36        """
37        spec = gf.builtin_specs()[0] # clean_single_rally
38        fps, fw = spec.fps, spec.frame_width
39        rows = gf.make_synthetic_trajectory(spec)
40        if frame_offset:
41            rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
42        gts = list(spec.gts)
43        if frame_offset:
44            off_sec = frame_offset / fps
45            gts = [(s + off_sec, e + off_sec) for (s, e) in gts]
46
47        traj = tmp_path / "src_trajectory.csv"
48        labels = tmp_path / "src_labels.csv"
49        gf.write_trajectory_csv(rows, traj)
50        gf.write_labels_csv(gts, labels)
51        return traj, labels, fps, fw, gts
52
53
54    # -----
55    # M2 ? refusal gate
56    # -----
57
58
59    @pytest.mark.parametrize(
60        "kwargs, why",
61        [
62            ({"minors_free": False, "owned": True, "license": "owned"},
"minors_free=False"),
63            ({"minors_free": True, "owned": False, "license": "owned"}, "owned=False"),
64            ({"minors_free": True, "owned": True, "license": ""}, "license=' '"),
65            ({"minors_free": True, "owned": True, "license": " "}, "license=blank"),

```

```

66         ({"minors_free": True, "owned": True, "license": None}, "license=None"),
67     ],
68 )
69 def test_refusal_gate_blocks_unsafe(tmp_path, kwargs, why):
70     """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
blank license.
71
72     The refusal is the code-enforced provenance gate (§4c / §7 ?): the unsafe public-
fixture path
73     must be impossible to reach silently.
74     """
75     traj, labels, fps, fw, _ = _write_source(tmp_path)
76     with pytest.raises(ValueError) as ei:
77         gf.freeze_real_fixture(
78             "scenario", traj, labels,
79             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
80     # RefusalError is a ValueError subclass; the message must name a reason.
81     assert isinstance(ei.value, gf.RefusalError), why
82     assert "REFUSED" in str(ei.value), why
83     # Nothing was written for a refused freeze.
84     assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
"fixtures").iterdir()), (
85         f"{why}: a refused freeze must not write any fixture")
86
87
88     # -----
89     # P1 ? de-attribution
90     # -----
91
92
93 def test_real_fixture_is_deattributed(tmp_path):
94     """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed
surrogate slug,
95     no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
96     fixtures = tmp_path / "fixtures"
97     traj, labels, fps, fw, _ = _write_source(tmp_path)
98     expected = gf.freeze_real_fixture(
99         "clean_rally", traj, labels,
100         license="owned", minors_free=True, owned=True,
101         fps=fps, frame_width=fw, fixtures_dir=fixtures)
102
103     slug = expected["slug"]
104     assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
{slug!r}"
105
106     prov = gf._read_json(fixtures / slug / "provenance.json")
107     assert prov["kind"] == "cleared_real"
108     assert prov["minors_free"] is True and prov["owned"] is True
109     assert prov["license"] == "owned"
110     assert prov["slug"] == slug
111     # No attribution keys / no source filename anywhere in the provenance.
112     for banned in ("video_id", "name", "filename", "path", "match", "capture_date"):
113         assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
114     prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
115     for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
116         assert tok not in prov_text, f"source token {tok!r} leaked into provenance"
117
118     # expected.json carries no biometric/identity/non-shuttle stream.
119     exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
120     for forbidden in gf.FORBIDDEN_KEYS:
121         assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"
122

```

```

123     # The fixture is registered in the manifest with kind=cleared_real.
124     manifest = gf.load_manifest(fixture)
125     rows = [e for e in manifest if e["slug"] == slug]
126     assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"
127
128
129     def test_supplied_slug_is_honored(tmp_path):
130         """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
131         concern)."""
132         fixtures = tmp_path / "fixtures"
133         traj, labels, fps, fw, _ = _write_source(tmp_path)
134         expected = gf.freeze_real_fixture(
135             "scenario", traj, labels, slug="fx_custom00",
136             license="owned", minors_free=True, owned=True,
137             fps=fps, frame_width=fw, fixtures_dir=fixtures)
138         assert expected["slug"] == "fx_custom00"
139         assert (fixtures / "fx_custom00" / "expected.json").is_file()
140
141     def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
142         """Freezing a second real fixture must APPEND to the manifest, not wipe the
143         first."""
144         fixtures = tmp_path / "fixtures"
145         traj, labels, fps, fw, _ = _write_source(tmp_path)
146         e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
147             license="owned", minors_free=True, owned=True,
148             fps=fps, frame_width=fw, fixtures_dir=fixtures)
149         e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
150             license="owned", minors_free=True, owned=True,
151             fps=fps, frame_width=fw, fixtures_dir=fixtures)
152         slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
153         assert {e1["slug"], e2["slug"]} <= slugs
154
155     def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
156         """source_note is non-identifying by contract ? a path/name in it trips the positive
157         scan."""
158         fixtures = tmp_path / "fixtures"
159         traj, labels, fps, fw, _ = _write_source(tmp_path)
160         with pytest.raises(gf.RefusalError):
161             gf.freeze_real_fixture(
162                 "scenario", traj, labels, slug="fx_leak0001",
163                 license="owned", minors_free=True, owned=True,
164                 fps=fps, frame_width=fw, fixtures_dir=fixtures,
165                 source_note=str(traj)) # leaking the source path ? must be refused
166
167     # -----
168     # P1 (d) ? time-origin reset is metric-invariant
169     # -----
170
171     def test_time_origin_reset_is_metric_invariant(tmp_path):
172         """A large-offset source frozen WITH reset has IDENTICAL score to the SAME source
173         frozen
174         WITHOUT reset ? but its absolute control_segments shift toward 0 (severed
175         timeline)."""
176         offset_frames = 9000 # +5 minutes at 30fps ? a large absolute capture-timeline
177         offset
178         traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)
179
180         with_reset = gf.freeze_real_fixture(
181             "scenario", traj, labels, slug="fx_reset_on",
182             license="owned", minors_free=True, owned=True,
183             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",

```

```

182         time_origin_reset=True)
183     without_reset = gf.freeze_real_fixture(
184         "scenario", traj, labels, slug="fx_reset_off",
185         license="owned", minors_free=True, owned=True,
186         fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
187         time_origin_reset=False)
188
189     # (1) Score is bit-identical ? metrics.score is built from pure differences.
190     assert with_reset["score"] == without_reset["score"], (
191         "time-origin reset must be metric-invariant (score built from pure
differences)")
192
193     # (2) Absolute timeline differs ? the reset pulls control_segments toward 0.
194     seg_on = with_reset["control_segments"]
195     seg_off = without_reset["control_segments"]
196     assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
197     assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
198     # The shift equals the offset in seconds (within rounding).
199     shift = seg_off[0][0] - seg_on[0][0]
200     assert abs(shift - offset_frames / fps) < 1e-3, (
201         f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
202     # Reset trajectory starts at frame 0.
203     first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
204                       .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
205     assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
206
207
208     # -----
209     # P1 (e/f) ? round-trip / correct-by-construction
210     # -----
211
212
213     def test_freeze_real_round_trips_check(tmp_path):
214         """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
215         returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
216         fixtures = tmp_path / "fixtures"
217         traj, labels, fps, fw, _ = _write_source(tmp_path)
218         expected = gf.freeze_real_fixture(
219             "scenario", traj, labels, slug="fx_rt000001",
220             license="owned", minors_free=True, owned=True,
221             fps=fps, frame_width=fw, fixtures_dir=fixtures)
222
223         fdir = fixtures / expected["slug"]
224         committed = gf._read_json(fdir / "expected.json")
225         recomputed = gf.replay_fixture(fdir)
226         assert gf.compare_expected(recomputed, committed) == [], (
227             "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
228
229
230     def test_freeze_real_uses_strict_label_loader(tmp_path):
231         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
232         never silently shrinks the GT set."""
233         fixtures = tmp_path / "fixtures"
234         traj, labels, fps, fw, _ = _write_source(tmp_path)
235         # Corrupt one label row (end <= start) ? strict loader must raise.
236         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
237         with pytest.raises(ValueError):
238             gf.freeze_real_fixture(
239                 "scenario", traj, labels, slug="fx_strict01",

```

```
240         license="owned", minors_free=True, owned=True,
241         fps=fps, frame_width=fw, fixtures_dir=fixtures)
242
```

## 7. user (2026-06-16T06:27:11.401Z)

```
1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10        filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11        active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12        -> rally windows.
13
14    Training labels come from Gemini full-context labels via the SAME IoU matching the
15    evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16    other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17    compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18    rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20    Run:
21        python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22    """
23    from __future__ import annotations
24
25    import json
26    import argparse
27    from typing import Dict, List, Optional, Tuple
28
29
30    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31    from backend.pipeline.detectors import rally_gate
32    from backend.eval.calibrate_wasb import load_golden_gts
33    from backend.eval.training import label_candidates
34    from backend.eval.classifier import LogisticRegression
35    from backend.eval.metrics import score
36    from backend.eval.gemini_refine import merge_overlaps
37    from backend.eval import windowing as _windowing
38
39    Interval = Tuple[float, float]
40
41    FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42    # Recall-oriented proposal: deliberately permissive so real rallies are among the
43    # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
44    # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
that
45    # module's docstring for the eval?serve bug these named presets close. The bare tuples
are
46    # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
47    # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
48    PROPOSAL = _windowing.PROPOSAL.as_tuple()
49    BASELINE_LOCAL = _windowing.SERVED.as_tuple() # the served (production) windowing, no
learning
50
51
52    def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
53        proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
```

```

54     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
55     points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
56     vt, mg, mwd = proposal
57     wins = TrackNetRunner.trajectory_to_action_windows(
58         points, fps=fps, frame_width=frame_width,
59         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
60     intervals = [(w["start"], w["end"]) for w in wins]
61     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
62     X: List[List[float]] = []
63     for w, g in zip(wins, gated):
64         dur = max(1e-6, w["end"] - w["start"])
65         win_frames = max(1.0, dur * fps)
66         X.append([
67             dur, # rally length (sec) ?
68             float(w["peak_velocity"]), # already normalized by
69             float(w["mean_velocity"]),
70             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
71             float(g.crossings), # net crossings (count) ?
72         ])
73     return intervals, X
74
75
76     def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
77         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
78         vt, mg, mwd = BASELINE_LOCAL
79         wins = TrackNetRunner.trajectory_to_action_windows(
80             points, fps=fps, frame_width=frame_width,
81             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
82         return [(w["start"], w["end"]) for w in wins]
83
84
85     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
86         fps, fw = float(spec["fps"]), float(spec["frame_width"])
87         intervals, X = build_candidates(spec["trajectory"], fps, fw)
88         gts = load_golden_gts(spec["labels"])
89         y = label_candidates(intervals, gts, iou)
90         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
91             "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}
92
93
94     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
95         threshold: float = 0.5) -> List[Interval]:
96         if not intervals:
97             return []
98         proba = model.predict_proba(X)
99         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
100         return merge_overlaps(pos, gap_tol=1.0)
101
102
103     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
104         model_out: Optional[str] = None) -> dict:
105         videos = [load_video(s, iou) for s in specs]
106         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
107             f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
108         for v in videos:

```



```

109         ceil = score(v["intervals"], v["gts"], iou).recall
110         print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
111               f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
112
113     result: dict = {"videos": [v["name"] for v in videos]}
114     if len(videos) >= 2:
115         print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
116         clf, base = [], []
117         for i, held in enumerate(videos):
118             others = [v for j, v in enumerate(videos) if j != i]
119             X = [row for v in others for row in v["X"]]
120             y = [t for v in others for t in v["y"]]
121             model = LogisticRegression().fit(X, y, FEATURE_NAMES)
122             preds = predict_rallies(model, held["X"], held["intervals"], threshold)
123             sc = score(preds, held["gts"], iou)
124             bsc = score(held["baseline"], held["gts"], iou)
125             clf.append(sc)
126             base.append(bsc)
127             print(f" held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
128                   f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
129         def mean(xs, a):
130             return sum(getattr(s, a) for s in xs) / len(xs)
131         print(f" MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f} "
132               f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
133               f"F1={mean(base, 'f1'):.3f} R={mean(base, 'recall'):.3f} "
134               f"F1={mean(base, 'f1'):.3f}")
135         result["mean_learned_f1"] = mean(clf, "f1")
136         result["mean_baseline_f1"] = mean(base, "f1")
137
138     # Final model on ALL videos (for production / running on a fresh held-out video).
139     Xall = [row for v in videos for row in v["X"]]
140     yall = [t for v in videos for t in v["y"]]
141     final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
142     assert final.w is not None # set by fit()
143     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
144     print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
145     if model_out:
146         final.save(model_out)
147         print(f"final model (all videos) -> {model_out}")
148         result["model_out"] = model_out
149     return result
150
151     def main() -> None:
152         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153         ap.add_argument("--manifest", required=True,
154                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155         ap.add_argument("--iou", type=float, default=0.5)
156         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157         ap.add_argument("--model-out", default="output/local_rally_model.json")
158         args = ap.parse_args()
159         with open(args.manifest, encoding="utf-8") as f:
160             specs = json.load(f)
161         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163     if __name__ == "__main__":
164         main()
165
166

```

## 8. assistant (2026-06-16T06:27:14.564Z)

Now let me read the detector files (tracknet\_runner, rally\_gate) and the other eval files to understand the trajectory parsing and what gets persisted.

## 9. user (2026-06-16T06:27:15.551Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True          # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"          # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800          # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False          # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0          # warn if WSL free space below this
(0 = off)
44
45      @classmethod
46      def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47          tn = (idx_cfg or {}).get("tracknet", {}) or {}
48          return cls(
49              distro=tn.get("wsl_distro", "Ubuntu"),
50              repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
```

```

51         conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52         conda_env=tn.get("conda_env", "TrackNetV4"),
53         weights_path=tn.get("weights_path", ""),
54         queue_length=int(tn.get("queue_length", 5)),
55         stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56         wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57         timeout_sec=int(tn.get("timeout_sec", 1800)),
58         keep_frames=bool(tn.get("keep_frames", False)),
59         min_free_gb=float(tn.get("min_free_gb", 0.0)),
60     )
61
62
63 def to_wsl_mnt_path(win_path: str) -> str:
64     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU'))))\"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "`wsl` not found ? is this running on Windows with WSL2 installed?"
112         except subprocess.TimeoutExpired:
113             return False, "WSL healthcheck timed out."
114         out = (res.stdout or "").strip()

```

```

114         if res.returncode != 0:
115             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
116         return True, out
117
118 # ----- inference
119
120 def run_predict(self, video_win_path: str, output_win_dir: str,
121                 video_id: Optional[str] = None) -> Optional[str]:
122     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
123     None.
124
125     ``output_win_dir`` is a Windows directory (under the repo's output/) that
126     WSL writes into via its /mnt view, so the Windows process can read the CSV
127     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
128     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
129     a filename cannot collide on the output CSV.
130     """
131     if not self.cfg.weights_path:
132         logger.error(
133             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
134             "in config.json to the WSL path of your trained TrackNetV4 weights."
135         )
136         return None
137
138     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
139     # relative paths through unchanged, so WSL would resolve them against the
140     # predict.py cwd instead of the repo (same bug class as wasb_runner).
141     output_win_dir = os.path.abspath(output_win_dir)
142     os.makedirs(output_win_dir, exist_ok=True)
143     # Normalize separators so basename/splitext work when tests (or collector)
144     # pass Windows-style paths on a Linux host.
145     video_win_path = str(video_win_path).replace("\\", "/")
146     base = os.path.basename(video_win_path)
147     stem, ext = os.path.splitext(base)
148
149     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
150     if ext.lower() not in (".mp4", ".avi"):
151         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
152         return None
153
154     wsl_out = to_wsl_mnt_path(output_win_dir)
155     # Namespace by video_id so two same-filename videos don't collide on the CSV.
156     # predict.py derives its output name from the (staged) video stem, so staging
157     # under the namespaced name propagates into "<key>_predict.csv".
158     key = f"{stem}__{video_id[:12]}" if video_id else stem
159     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
160
161     # Resolve the video path WSL will read.
162     if self.cfg.stage_in_wsl:
163         staged = self._stage_video(video_win_path, f"{key}{ext}")
164         if staged is None:
165             return None
166         wsl_video = staged
167     else:
168         # No staging: predict.py reads /mnt directly and writes
169         # "<stem>_predict.csv".
170         # We cannot rename its output, so fall back to the un-namespaced name.
171         wsl_video = to_wsl_mnt_path(video_win_path)
172         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
173
174     if self.cfg.min_free_gb > 0:
175         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
176         if free is not None and free < self.cfg.min_free_gb:
177             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
178                           "consider running tools/cleanup_caches.py.",

```

```

177         free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179     cmd = (
180         f"conda activate {self.cfg.conda_env} && "
181         f"cd {self.cfg.repo_dir}/src && "
182         f"python predict.py "
183         f"--video_path {wsl_tilde_quote(wsl_video)} "
184         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185         f"--output_dir {wsl_tilde_quote(wsl_out)} "
186         f"--queue_length {self.cfg.queue_length}"
187     )
188     logger.info("Running TrackNet inference on %s ...", base)
189     try:
190         res = self._wsl_bash(cmd)
191     except subprocess.TimeoutExpired:
192         logger.error("TrackNet inference timed out after %ss.",
193 self.cfg.timeout_sec)
194         return None
195
196     if res.returncode != 0:
197         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
198 res.stdout)[-2000:])
199         return None
200
201     if not os.path.exists(expected_csv_win):
202         logger.error("TrackNet finished but expected CSV not found at %s",
203 expected_csv_win)
204         return None
205     logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
206
207     # Success ? the CSV is the durable output; drop the staged video copy (a pure,
208     # regenerable intermediate). On earlier failure we return above without
209     cleaning.
210     if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
211         logger.info("Cache hygiene: removing staged video for %s "
212             "(set indexing.tracknet.keep_frames=true to retain).", key)
213         self._wsl_rm_rf([wsl_video])
214     return expected_csv_win
215
216 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
217     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
218
219     Returns the WSL path to the staged copy, or None on failure.
220     """
221     src_mnt = to_wsl_mnt_path(video_win_path)
222     reason = self.wsl_source_unreadable_reason(src_mnt)
223     if reason:
224         logger.error("Cannot stage video into WSL: %s", reason)
225         return None
226     dest = f"{self.cfg.wsl_stage_dir}/{base}"
227     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
228 {wsl_tilde_quote(dest)} && echo OK"
229     try:
230         res = self._wsl_bash(cmd)
231     except subprocess.TimeoutExpired:
232         logger.error("Staging video into WSL timed out.")
233         return None
234     if res.returncode != 0 or "OK" not in (res.stdout or ""):
235         logger.error("Failed to stage video into WSL: %s", (res.stderr or
236 res.stdout).strip())
237         return None
238     return dest
239
240 # ----- CSV -> windows

```

```

236 @staticmethod
237 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239     points: List[TrajectoryPoint] = []
240     with open(csv_win_path, newline="") as f:
241         reader = csv.DictReader(f)
242         for row in reader:
243             try:
244                 points.append(TrajectoryPoint(
245                     frame=int(row["Frame"]),
246                     visible=int(row["Visibility"]) == 1,
247                     x=float(row["X"]),
248                     y=float(row["Y"]),
249                 ))
250             except (KeyError, ValueError):
251                 continue
252     points.sort(key=lambda p: p.frame)
253     return points
254
255 @staticmethod
256 def trajectory_to_action_windows(
257     points: List[TrajectoryPoint],
258     fps: float,
259     frame_width: float,
260     chunk_start: float = 0.0,
261     velocity_thresh: float = 0.02,
262     merge_gap: float = 2.0,
263     min_window_duration: float = 2.0,
264     chunk_index: Optional[int] = None,
265     max_window_duration: float = 0.0,
266     min_split_gap: float = 0.5,
267 ) -> List[Dict[str, Any]]:
268     """Convert a shuttle trajectory into candidate rally windows.
269
270     A frame is "active" when the shuttle is visible AND its inter-frame
271     displacement (normalised by frame width, per second) exceeds
272     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
273     ``merge_gap`` seconds of each other are grouped into one window; short
274     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
275     shape feeding the AI-handoff phase is identical.
276
277     ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
278     against
279     the *over-merge* failure where "shuttle moving" is conflated with "rally in
280     progress"
281     and a single window swallows several rallies + the dead time between them. When
282     set, any
283     window longer than this is recursively SPLIT at its largest internal inactivity
284     gap (the
285     longest stretch with no in-flight shuttle ? the most likely rally boundary),
286     provided
287     that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
288     fix from
289     ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
290     recall cannot
291     drop, and it is config-gated default-OFF until measured on the golden set.
292     """
293     if fps <= 0:
294         fps = 30.0
295     if frame_width <= 0:
296         frame_width = 1280.0
297
298     # Compute per-frame normalised velocity from the last *visible* point.
299     active = [] # (frame_idx, velocity)
300     prev: Optional[TrajectoryPoint] = None

```

```

294     for p in points:
295         if not p.visible:
296             prev = None # break the trajectory; absent shuttle = not in flight
297             continue
298         if prev is not None:
299             dframes = p.frame - prev.frame
300             if dframes > 0:
301                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
302                 velocity = (dist / frame_width) * (fps / dframes)
303                 if velocity > velocity_thresh:
304                     active.append((p.frame, velocity))
305             prev = p
306
307     if not active:
308         return []
309
310     max_gap_frames = int(merge_gap * fps)
311
312     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
313         vels = [v for _, v in group]
314         return {
315             "start": group[0][0] / fps + chunk_start,
316             "end": group[-1][0] / fps + chunk_start,
317             "active_frame_count": len(group),
318             "peak_velocity": max(vels),
319             "mean_velocity": sum(vels) / len(vels),
320             "track_id_count": 1, # single shuttle; kept for window-shape parity
321             "chunk_index": chunk_index,
322         }
323
324     groups: List[List[Tuple[int, float]]] = []
325     group = [active[0]]
326     for af in active[1:]:
327         if af[0] - group[-1][0] <= max_gap_frames:
328             group.append(af)
329         else:
330             groups.append(group)
331             group = [af]
332     groups.append(group)
333
334     if max_window_duration > 0:
335         groups = TrackNetRunner._split_overlong_groups(
336             groups, int(max_window_duration * fps), int(min_split_gap * fps))
337
338     windows = [_finalize(g) for g in groups]
339     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
340
341     @staticmethod
342     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
343                               min_split_frames: int) -> List[List[Tuple[int, float]]]:
344         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
345         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
346         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
347         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
348         rally). Iterative worklist to bound stack depth."""
349         if max_win_frames <= 0:
350             return groups
351         out: List[List[Tuple[int, float]]] = []
352         work = list(groups)
353         while work:

```

```

354         g = work.pop()
355         if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
356             out.append(g)
357             continue
358         best_i, best_gap = -1, -1
359         for i in range(1, len(g)):
360             gap = g[i][0] - g[i - 1][0]
361             if gap > best_gap:
362                 best_gap, best_i = gap, i
363         if best_i < 0 or best_gap < min_split_frames:
364             out.append(g) # too long but no real dead-time gap ? keep as one rally
365             continue
366         work.append(g[:best_i])
367         work.append(g[best_i:])
368     out.sort(key=lambda grp: grp[0][0])
369     return out
370

```

## 10. user (2026-06-16T06:27:16.090Z)

```

1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5       ``curate export`` format), raises on bad rows;
6     - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13
14    Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15    - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16      columns ? rally_number, ending_reason, sport, ? ? are ignored);
17    - no header: columns 0,1 positionally.
18    Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19    (``annotations.parse_timestamp``).
20
21    """
22    import csv
23    import logging
24    from typing import List, Optional, Tuple
25
26    from backend.eval.annotations import parse_timestamp
27
28    logger = logging.getLogger(__name__)
29
30    Interval = Tuple[float, float]
31
32    _START_NAMES = ("start", "start_time")
33    _END_NAMES = ("end", "end_time")
34
35    def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
36        """If `row` is a header naming start/end columns, return their indices, else
37        None."""
38        cells = [(c or "").strip().lower() for c in row]
39        start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
40        end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
41        if start_idx is not None and end_idx is not None:
42            return start_idx, end_idx
43        return None
44

```



```

43
44 def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45     """Load GT rally intervals from either golden-CSV convention.
46
47     strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48     (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49     strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50     skipped ? a silently shrinking GT set flatters precision.
51
52     Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53     (they break one-to-one matching and usually indicate a labeling error).
54     """
55     intervals: List[Interval] = []
56     skipped = 0
57     cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
58     saw_header = False
59
60     with open(csv_path, newline="", encoding="utf-8-sig") as f:
61         for lineno, row in enumerate(csv.reader(f), start=1):
62             if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                 continue
64             if cols is None and not saw_header:
65                 header = _header_columns(row)
66                 if header is not None:
67                     cols = header
68                     saw_header = True
69                     continue # consume the header row
70             cols = (0, 1) # headerless: positional start,end
71             assert cols is not None # every branch above assigns it
72             si, ei = cols
73             try:
74                 if len(row) <= max(si, ei):
75                     raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                 start = parse_timestamp(row[si])
77                 end = parse_timestamp(row[ei])
78                 if start < 0:
79                     raise ValueError(f"negative start ({start})")
80                 if start >= end:
81                     raise ValueError(f"start ({start}) >= end ({end})")
82             except ValueError as e:
83                 if strict:
84                     raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                 skipped += 1
86                 continue
87             intervals.append((start, end))
88
89     if skipped:
90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                 "labeling error? Overlaps break one-to-one matching.",
97                 csv_path, s1, e1, s2, e2)
98     return intervals
99

```

## 11. assistant (2026-06-16T06:27:18.845Z)

Now let me read the experiment, metrics, regression, and rally\_gate files, plus the committed fixtures and CI workflow.

## 12. user (2026-06-16T06:27:20.268Z)

```
1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. No new scoring math lives
7      here; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`   ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
56     ratio
57     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
58     scores
59
60     logger = logging.getLogger(__name__)
61
62     # Where a comparison run appends one reproducible record. Tracked location (NOT under
63     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
```

```

62 # regression.DEFAULT_BASELINE_PATH.
63 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64 _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67 class ExperimentError(ValueError):
68     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69     score an unlabeled video with no pinned model, or a gate referencing an absent
70     feature)."""
71
72
73 # ----- Variant
74
75
76 @dataclass
77 class Variant:
78     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80     A variant turns one video's persisted candidate windows + features into rally
81     intervals. Every knob defaults to the control behaviour (no gate, no learned
82     filter), so a variant that merely carries a new, disabled cue is byte-identical
83     to control ? the core no-regression guarantee.
84
85     Fields:
86     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87       for the leaderboard and for selecting the served path (the existing
88       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89       New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for ``compare_unlabeled`` on a learned variant.
100     - ``feature_names`` ? the persisted features the learned filter consumes. When set
101       WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102       (honest) ? the recipe, not a pinned artifact.
103     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104       overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105     """
106
107     name: str
108     segmenter: str = "wasb_hybrid"
109     config_overrides: Dict[str, Any] = field(default_factory=dict)
110     cue_flags: Dict[str, bool] = field(default_factory=dict)
111     min_crossings: int = 0
112     min_players: int = 0
113     classifier_model: Optional[str] = None
114     feature_names: Optional[List[str]] = None
115     threshold: float = 0.5
116     merge_gap: float = 1.0
117     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None # None | "platt" | "isotonic" ? calibrate
proba first
122

```

```

123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,
138             "merge_gap": self.merge_gap,
139             "tune_threshold": self.tune_threshold,
140             "calibrate": self.calibrate,
141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
the
151         same, so a regression is always traceable to a recipe change, never to drift."""
152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:

```

```

186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                         name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196         cols = [_feature_column(vc, name) for name in feature_names]
197         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202         players)."""
203         n = len(vc.intervals)
204         mask = [True] * n
205         if variant.min_crossings > 0:
206             col = _feature_column(vc, "net_crossings")
207             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208         if variant.min_players > 0:
209             col = _feature_column(vc, "players_in_court")
210             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211         return mask
212
213     def predict(variant: Variant, vc: VideoCandidates,
214                 model: Optional[LogisticRegression] = None,
215                 threshold: Optional[float] = None,
216                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217         """Variant prediction: gate by persisted features, optionally filter by a learned
218         model, then merge. Pure and deterministic.
219
220         ``model`` (an already-fitted ``LogisticRegression``) is supplied by ``compare`` per LOVO
221         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225         them.
226
227         """
228         mask = _gate_mask(variant, vc)
229         if variant.feature_names:
230             if model is None:
231                 if variant.classifier_model is None:
232                     raise ExperimentError(
233                         f"variant {variant.name!r} is learned but has no model to predict "
234                         f"with (pass a fitted model or set classifier_model)")
235                 model = LogisticRegression.load(variant.classifier_model)
236             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
237 variant.feature_names))]
238             if calibrator is not None:
239                 proba = [calibrator(p) for p in proba]
240             thr = variant.threshold if threshold is None else threshold
241             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
242             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
243             return merge_overlaps(kept, gap_tol=variant.merge_gap)
244
245     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
246                             train_videos: Sequence[VideoCandidates], iou: float
247                             ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":

```

```

246     """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247     (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
248     (use the variant default). Honest: never looks at the held-out video.
249     """
250     calibrator: Optional[Callable[[float], float]] = None
251     if variant.calibrate:
252         raw: List[float] = []
253         y: List[int] = []
254         for vc in train_videos:
255             raw.extend(float(p) for p in
256                         model.predict_proba(_feature_matrix(vc, variant.feature_names or
257                                                         [])))
258             y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
259         if variant.calibrate == "platt":
260             calibrator = fit_platt(raw, y)
261         elif variant.calibrate == "isotonic":
262             calibrator = fit_isotonic(raw, y)
263         else:
264             raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
265                                 'platt'/'isotonic')")
266     threshold: Optional[float] = None
267     if variant.tune_threshold:
268         best_t, best_f1 = variant.threshold, -1.0
269         for k in range(1, 20):
270             # grid 0.05..0.95 on the train
271             t = round(0.05 * k, 2)
272             f1s = [score(predict(variant, vc, model=model, threshold=t,
273                                 calibrator=calibrator),
274                     vc.gts or [], iou).f1 for vc in train_videos]
275             mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
276             if mean_f1 > best_f1:
277                 best_f1, best_t = mean_f1, t
278             threshold = best_t
279     return calibrator, threshold
280
281 # ----- variant scoring
282
283 def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
284 LogisticRegression:
285     """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
286     the
287     evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
288     X: List[List[float]] = []
289     y: List[int] = []
290     for vc in videos:
291         if vc.gts is None:
292             raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
293         X.extend(_feature_matrix(vc, variant.feature_names or []))
294         y.extend(label_candidates(vc.intervals, vc.gts, iou))
295     if not X:
296         raise ExperimentError("cannot train a learned variant on zero candidates")
297     return LogisticRegression().fit(X, y, variant.feature_names)
298
299 def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
300                     iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
301     """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
302     mean aggregate.
303
304     Prediction policy:
305     - **static variant** (no learned filter): predict each video directly ? no
306     training,
307     so no leakage; per-video scoring is already honest.

```

```

304         - **learned, pinned model** (``classifier_model`` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
306         to
307         report "how the shipped artifact does here", not for the promotion decision.
308         - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
309         ?
310         train on the OTHER videos, predict the held-out one. The number to trust.
311         - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
312         only).
313         """
314         for vc in videos:
315             if vc.gts is None:
316                 raise ExperimentError(f"{vc.name} has no golden labels; use
317                 compare_unlabeled")
318             per_video: List[Dict[str, Any]] = []
319             predictions: Dict[str, List[Interval]] = {}
320             recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
321             pinned_model = (LogisticRegression.load(variant.classifier_model)
322                             if variant.classifier_model is not None else None)
323             all_model = None
324             if recipe_lovo and not honest:
325                 all_model = _train(variant, videos, iou)
326             for i, vc in enumerate(videos):
327                 cal: Optional[Callable[[float], float]] = None
328                 thr: Optional[float] = None
329                 if recipe_lovo and honest:
330                     others = [v for j, v in enumerate(videos) if j != i]
331                     if not others:
332                         raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
333                         pinned model")
334                     model: Optional[LogisticRegression] = _train(variant, others, iou)
335                     if variant.tune_threshold or variant.calibrate: # operating point from
336                         the train fold
337                             assert model is not None # _train never returns
338                             None
339                             cal, thr = _calibrate_and_tune(variant, model, others, iou)
340                     elif recipe_lovo: # honest=False ? one model over all
341                         videos
342                         model = all_model
343                     else: # static variant or pinned model
344                         model = pinned_model
345                         preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
346                         assert vc.gts is not None # guarded at function top
347                         sc = score(preds, vc.gts, iou)
348                         per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
349                         for m in _METRICS}})
350                         predictions[vc.name] = preds
351             aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
352             0.0)
353             for m in _METRICS}
354             return {
355                 "variant": variant.name,
356                 "spec_hash": variant.spec_hash(),
357                 "is_learned": variant.is_learned(),
358                 "honest_lovo": recipe_lovo and honest,
359                 "per_video": per_video,
360                 "aggregate": aggregate,
361                 "predictions": predictions,
362             }

```

```

359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
eval_b: Dict[str, Any]) -> Dict[str, Any]:
385         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS $6).
386
387         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
388         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
389         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
390         """
391         a_f1 = [p["f1"] for p in eval_a["per_video"]]
392         b_f1 = [p["f1"] for p in eval_b["per_video"]]
393         return {
394             "variant_a_ci": _ci_block(eval_a),
395             "variant_b_ci": _ci_block(eval_b),
396             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
397             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
398                             "variant_b": _segmentation_block(videos, eval_b)},
399         }
400
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` $5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.

```



```

411
412     ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413     paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414     existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415     shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416     bare LOVO mean at n?6 is not trustworthy on its own.
417     ""
418     if len(videos) < 1:
419         raise ExperimentError("compare needs at least one labeled video")
420     eval_a = evaluate_variant(videos, variant_a, iou, honest)
421     eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427     per_video_delta = [
428         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429         for p in eval_b["per_video"]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451     # ----- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455         agree_iou: float = 0.5) -> Dict[str, Any]:
456         ""Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
465           the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
a

```

```

468 pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469 `metrics.match_intervals` ? no new matching logic.
470 """
471 preds_a = predict(variant_a, video)
472 preds_b = predict(variant_b, video)
473 mr = match_intervals(preds_a, preds_b, agree_iou)
474
475 agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
476           for m in mr.matches]
477 agree_iou = [m.iou for m in mr.matches]
478 a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
479 b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
480
481 def _dur(ivs: List[Interval]) -> float:
482     return sum(e - s for s, e in ivs)
483
484 return {
485     "video": video.name,
486     "agree_iou": agree_iou,
487     "variant_a": variant_a.name,
488     "variant_b": variant_b.name,
489     "num_a": len(preds_a),
490     "num_b": len(preds_b),
491     "num_agreed": len(agreed),
492     "mean_agree_iou": (sum(agree_iou) / len(agree_iou)) if agree_iou else 0.0,
493     "duration_a": _dur(preds_a),
494     "duration_b": _dur(preds_b),
495     "agreed": agreed,
496     "a_only": a_only,
497     "b_only": b_only,
498 }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:

```

```

532         entries = json.load(f)
533     entries.append(row)
534     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535     with open(path, "w", encoding="utf-8") as f:
536         json.dump(entries, f, indent=2)
537     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538                 row["variant_a"]["name"], row["variant_b"]["name"],
539                 (result.get("delta") or {}).get("f1", 0.0), path)
540     return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
547         `Database.query_candidate_segments`.
548         ``feature_names`` selects which persisted features to lift out of each row's
549         ``stats``
550         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551         touching
552         function; the comparison itself is pure and offline. Exercised owner-side on a real
553         DB,
554         but works against any `Database` (including an in-memory one in tests).
555         """
556         rows = db.query_candidate_segments(video_id, detector=detector)
557         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558                                     rows]
559         features: Dict[str, List[float]] = {}
560         for name in feature_names:
561             if name == "duration":
562                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
563                                 rows]
564             else:
565                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
566                                 rows]
567         return VideoCandidates(name=video_id, intervals=intervals, features=features,
568                                gts=gts)
569
570

```

### 13. user (2026-06-16T06:27:20.746Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19        overlap / union, both measured as durations on the timeline.
20        """
21        a_start, a_end = a
22        b_start, b_end = b
23        if a_end <= a_start or b_end <= b_start:

```

```

24         return 0.0
25     inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26     if inter <= 0.0:
27         return 0.0
28     union = (a_end - a_start) + (b_end - b_start) - inter
29     return inter / union if union > 0 else 0.0
30
31
32 @dataclass
33 class Match:
34     """One matched predicted?ground-truth pair."""
35     pred_index: int
36     gt_index: int
37     iou: float
38
39
40 @dataclass
41 class MatchResult:
42     """Outcome of matching predictions to ground truth at a given IoU threshold."""
43     matches: List[Match] = field(default_factory=list)
44     unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45     unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
46 (misses)
47
48 def match_intervals(preds: List[Interval], gts: List[Interval],
49                    iou_threshold: float = 0.5) -> MatchResult:
50     """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52     Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53     are claimed highest-IoU first, and each prediction/GT can be used once. This
54     keeps a single prediction from being credited for two ground-truth rallies
55     (and vice versa).
56     """
57     candidates = []
58     for pi, p in enumerate(preds):
59         for gi, g in enumerate(gts):
60             iou = interval_iou(p, g)
61             # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
62             # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63             if iou > 0.0 and iou >= iou_threshold:
64                 candidates.append((iou, pi, gi))
65     # Highest IoU first; ties broken by indices for determinism.
66     candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68     used_pred, used_gt = set(), set()
69     matches: List[Match] = []
70     for iou, pi, gi in candidates:
71         if pi in used_pred or gi in used_gt:
72             continue
73         used_pred.add(pi)
74         used_gt.add(gi)
75         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79     return MatchResult(matches=matches,
80                        unmatched_pred_indices=unmatched_pred,
81                        unmatched_gt_indices=unmatched_gt)
82
83
84 @dataclass
85 class Score:
86     """Aggregate metrics for one (predictions vs ground-truth) evaluation."""

```

```

87     iou_threshold: float
88     num_pred: int
89     num_gt: int
90     true_positives: int
91     false_positives: int
92     false_negatives: int
93     precision: float
94     recall: float
95     f1: float
96     mean_iou: float          # over matched pairs (0.0 if none)
97     mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
98     mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
99
100     def as_dict(self) -> dict:
101         return self.__dict__.copy()
102
103
104     def score(preds: List[Interval], gts: List[Interval],
105              iou_threshold: float = 0.5,
106              match_result: Optional[MatchResult] = None) -> Score:
107         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110         extracting the failure lists); otherwise matching is done here.
111         """
112         mr = match_result if match_result is not None else match_intervals(preds, gts,
113 iou_threshold)
114
115         tp = len(mr.matches)
116         fp = len(mr.unmatched_pred_indices)
117         fn = len(mr.unmatched_gt_indices)
118
119         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
120         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
121         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
122     else 0.0
123
124         if mr.matches:
125             mean_iou = sum(m.iou for m in mr.matches) / tp
126             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
127 mr.matches) / tp
128             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
129 mr.matches) / tp
130         else:
131             mean_iou = mean_start_err = mean_end_err = 0.0
132
133         return Score(
134             iou_threshold=iou_threshold,
135             num_pred=len(preds), num_gt=len(gts),
136             true_positives=tp, false_positives=fp, false_negatives=fn,
137             precision=precision, recall=recall, f1=f1,
138             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
139         )
140
141
142     def pr_curve(preds: List[Interval], gts: List[Interval],
143                 thresholds: Optional[List[float]] = None) -> List[Score]:
144         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
145
146         Lets you see strict-vs-lenient behaviour rather than a single operating point.
147         """
148         if thresholds is None:
149             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
150         return [score(preds, gts, iou_threshold=t) for t in thresholds]

```

#### 14. user (2026-06-16T06:27:21.051Z)

```
1      """Regression runner ? verify the production tool against golden-set ground truth.
2
3      Implements the explicit ask: *"given a video, get a golden set created and verify
4      its precision and recall against the production tool."* This is deliberately a thin
5      wrapper over machinery that already exists:
6
7      ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8      predictions  ? the production pipeline's stored DB rows    [backend.eval.harness]
9      comparison   ? interval matching + P/R/F1                  [backend.eval.metrics]
10     baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12     A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13     baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14     later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15     independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
39         different labels."""
40         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
41         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
59         latter.
60         has_baseline: bool = False
61         gt_hash: Optional[str] = None
```

```

62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78
79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                      f1: float, path: str = DEFAULT_BASELINE_PATH,
82                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                      gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87         later run computed under different settings (or against different labels) is
88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }
96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100                    _baseline_key(video_id, stage), precision, recall)
101
102     # ----- the runner
103
104     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105               stage: str = "final", iou_threshold: float = 0.5,
106               detector: Optional[str] = None,
107               tolerance: float = DEFAULT_TOLERANCE,
108               baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109         """Score stored predictions for one video against ground truth and compare to
110         baseline.
111
112         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
113         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
114         precision OR recall falls more than `tolerance` below the snapshotted baseline.
115         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
116         caller can choose to snapshot the current numbers via `save_baseline`.
117         """
118         preds = get_predictions(db, video_id, stage, detector=detector)
119         s = metrics.score(preds, ground_truth, iou_threshold)
120         fp = gt_hash(ground_truth)
121
122         baselines = load_baselines(baseline_path)
123         b = baselines.get(_baseline_key(video_id, stage))

```

```

124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
132         trusting it.
133         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
134         b.get("gt_hash")
135         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
136             regressed = True
137             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
138         if b_det is not None and b_det != detector:
139             regressed = True
140             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
141         if b_hash is not None and b_hash != fp:
142             regressed = True
143             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
144         if s.precision < base_p - tolerance:
145             regressed = True
146             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
147         if s.recall < base_r - tolerance:
148             regressed = True
149             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
150     else:
151         reasons.append("no baseline recorded for this video/stage ? nothing to compare")
152
153     return RegressionResult(
154         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
155         precision=s.precision, recall=s.recall, f1=s.f1,
156         baseline_precision=base_p, baseline_recall=base_r,
157         tolerance=tolerance, regressed=regressed, reasons=reasons,
158         has_baseline=has_baseline, gt_hash=fp,
159     )
160
161 def regress_with_provider(db: Database, video_id: str, video_ref: str,
162     tier: str = "file", stage: str = "final",
163     iou_threshold: float = 0.5,
164     detector: Optional[str] = None,
165     tolerance: float = DEFAULT_TOLERANCE,
166     baseline_path: str = DEFAULT_BASELINE_PATH,
167     guideline: Optional[str] = None,
168     sport: str = "badminton",
169     provider_kwargs: Optional[dict] = None) -> RegressionResult:
170     """End-to-end: obtain ground truth from the `tier` annotation provider, then
171     regress.
172     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
173     once those providers land). `video_ref` is whatever that provider expects (a CSV
174     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
175     predictions in the DB (the file MD5).
176     """
177     provider = annotation_registry.get(tier, **(provider_kwargs or {}))
178     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
179     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
180         detector=detector, tolerance=tolerance, baseline_path=baseline_path)

```



```

180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                 f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + "".join(f"\n    - {why}" for why in r.reasons)
189

```

## 15. assistant (2026-06-16T06:27:24.054Z)

Now let me read the rally\_gate, the committed fixtures, the CI workflow, and the synthetic test file.

## 16. user (2026-06-16T06:27:25.434Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Optional, Sequence, Tuple
24
25    Interval = Tuple[float, float]
26
27
28    @dataclass
29    class GatedWindow:
30        start: float
31        end: float
32        crossings: int
33        visible_points: int
34        kept: bool
35
36
37    def _spread(vals: Sequence[float]) -> float:
38        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39        if len(vals) < 2:
40            return 0.0
41        s = sorted(vals)
42        lo = s[max(0, int(0.05 * (len(s) - 1)))]
43        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44        return hi - lo
45
46

```



```

110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

## 17. user (2026-06-16T06:27:25.903Z)

```

1     """Golden Regression Fixtures (M1) ? characterization tests over the committed synthetic
Tier-0.
2
3     Parametrized over ``load_manifest()`` so every committed fixture is exercised. The gate
is
4     **parse-compare** (never byte-compare), **CONTROL-only** (pure stdlib, no numpy on the
replay
5     path), and asserts the Tier-0 privacy invariant positively (no
person/audio/pose/gait/face keys).
6
7     §6 honest-limits caveat: green here proves only that the deterministic post-trajectory
transforms
8     are unchanged for the frozen synthetic cases ? characterization is behaviour-locking,
NOT
9     correctness, and synthetic-tier numbers measure plumbing, not field quality.
10    """
11    import json
12
13    import pytest
14
15    from backend.eval import golden_fixtures as gf
16
17    # Slugs discovered from the committed manifest ? the suite grows by accretion
automatically.
18    MANIFEST = gf.load_manifest()
19    SLUGS = [entry["slug"] for entry in MANIFEST]
20
21
22    def test_manifest_is_non_empty():
23        """A committed manifest with at least one fixture must exist (else nothing is
gated)."""
24        assert SLUGS, "no fixtures in eval_baselines/fixtures/manifest.json ? run --freeze-
synthetic"
25
26
27    @pytest.mark.parametrize("slug", SLUGS)
28    def test_replay_matches_committed_expected(slug):
29        """Replay the fixture and parse-compare against its committed expected.json.
30
31        Characterization = behaviour-locking, NOT correctness (§6): a non-empty mismatch
list means
32        the deterministic post-trajectory behaviour CHANGED for this frozen case, not that
it is wrong.
33        """
34        committed = gf.load_fixture(slug)["expected"]
35        recomputed = gf.replay_fixture(slug)
36        mismatches = gf.compare_expected(recomputed, committed)
37        assert mismatches == [], (
38            f"{slug}: behaviour changed vs frozen snapshot (characterization, not
correctness ? §6):\n"
39            + "\n".join(f"  - {m}" for m in mismatches)
40        )
41

```

```

42
43     @pytest.mark.parametrize("slug", SLUGS)
44     def test_schema_version_le_current(slug):
45         """Every committed fixture's schema_version must be <= CURRENT_SCHEMA (forward-
compatible)."""
46         fx = gf.load_fixture(slug)
47         assert int(fx["expected"]["schema_version"]) <= gf.CURRENT_SCHEMA
48         assert int(fx["provenance"]["schema_version"]) <= gf.CURRENT_SCHEMA
49
50
51     @pytest.mark.parametrize("slug", SLUGS)
52     def test_no_person_or_audio_keys(slug):
53         """Positive privacy assertion (D3 / §4c): NO biometric/identity/non-shuttle key
appears
54         anywhere in the committed expected.json ? not trusting a silent 0.0 default."""
55         fdir = gf.FIXTURES_DIR / slug
56         raw = (fdir / "expected.json").read_text(encoding="utf-8").lower()
57         for forbidden in gf.FORBIDDEN_KEYS:
58             assert forbidden not in raw, (
59                 f"{slug}: forbidden key substring {forbidden!r} present in expected.json
(Tier-0 leak)")
60
61         expected = gf.load_fixture(slug)["expected"]
62         for i, cand in enumerate(expected["candidates"]):
63             # The only feature block is the shuttle block, and it holds EXACTLY the 5
shuttle cols.
64             assert set(cand.keys()) == {"start", "end", "shuttle"}, f"{slug}: candidate[{i}]
has extra keys"
65             assert set(cand["shuttle"].keys()) == set(gf.SHUTTLE_FEATURES), (
66                 f"{slug}: candidate[{i}].shuttle is not exactly the 5 shuttle features")
67
68
69     @pytest.mark.parametrize("slug", SLUGS)
70     def test_synthetic_provenance_tags(slug):
71         """Tier-0 hard tags: kind=synthetic, license=synthetic, minors_free=true (constraint
5)."""
72         prov = gf.load_fixture(slug)["provenance"]
73         assert prov["kind"] == "synthetic"
74         assert prov["license"] == "synthetic"
75         assert prov["minors_free"] is True
76         # Windowing pinned in the fixture (not the named SERVED constant) ? constraint 3.
77         assert set(prov["windowing"]) == {"vt", "mg", "mwd", "max_win"}
78
79
80     def test_control_is_not_learned():
81         """The replay must use the pure-stdlib CONTROL variant (no numpy/LogisticRegression
? D5)."""
82         from backend.eval.experiment import CONTROL
83         assert CONTROL.is_learned() is False
84
85
86     def test_perturbing_trajectory_changes_segments(tmp_path):
87         """Mutating a copy of a trajectory and replaying yields DIFFERENT segments ? proving
the
88         windows are RE-DERIVED from the CSV each run, not merely echoed from
expected.json."""
89         slug = SLUGS[0]
90         src = gf.FIXTURES_DIR / slug
91         work = tmp_path / slug
92         work.mkdir()
93         # Copy the fixture, then TRUNCATE the trajectory to its first half so the derived
window's
94         # `end` shrinks ? a guaranteed structural change in the re-derived segments.
(Deleting an
95         # interior hole would not move start/end, since merge_gap coalesces across it and

```

```

the frame
96         # indices of the surviving rows are unchanged.)
97         lines = (src / "trajectory.csv").read_text(encoding="utf-8").splitlines()
98         header, body = lines[0], lines[1:]
99         mutated = [header] + body[: len(body) // 2]
100        (work / "trajectory.csv").write_text("\n".join(mutated) + "\n", encoding="utf-8",
        newline="\n")
101        (work / "labels.csv").write_text((src / "labels.csv").read_text(encoding="utf-8"),
102                                         encoding="utf-8", newline="\n")
103        (work / "provenance.json").write_text((src /
        "provenance.json").read_text(encoding="utf-8"),
104                                              encoding="utf-8", newline="\n")
105
106        original = gf.load_fixture(slug)["expected"]
107        perturbed = gf.replay_fixture(work)
108        assert perturbed["control_segments"] != original["control_segments"], (
109            "perturbed trajectory produced identical segments ? replay is echoing, not re-
        deriving")
110
111
112    def test_freeze_is_idempotent(tmp_path):
113        """Re-freezing every built-in fixture into a temp dir produces expected.json
        structurally
114        equal to the committed one (correct-by-construction + deterministic)."""
115        slugs = gf.freeze_synthetic(fixtures_dir=tmp_path)
116        assert slugs, "freeze_synthetic produced no fixtures"
117        for slug in slugs:
118            committed = gf.load_fixture(slug)["expected"]
119            refrozen = json.loads((tmp_path / slug /
        "expected.json").read_text(encoding="utf-8"))
120            # Structural (parsed) equality ? no float churn, exact same shape/values.
121            assert gf.compare_expected(refrozen, committed) == [], (
122                f"{slug}: re-frozen expected.json differs from committed (non-idempotent
        freeze)")
123            assert refrozen == committed, f"{slug}: re-frozen expected.json not structurally
        identical"
124
125
126    def test_check_cli_passes(capsys):
127        """The --check CLI replays all fixtures and exits 0 when they match the committed
        snapshots."""
128        rc = gf.main(["--check"])
129        assert rc == 0
130

```

## 18. assistant (2026-06-16T06:27:28.579Z)

Now let me look at the committed fixtures and the CI workflow. Let me find the actual fixture files.

## 19. user (2026-06-16T06:27:31.379Z)

```

total 28
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 .
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 ..
-rw-r--r-- 1 avidu 197609 4018 Jun 16 11:54 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 clean_single_rally
-rw-r--r-- 1 avidu 197609 1557 Jun 16 11:54 manifest.json
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 multi_rally_with_deadtime
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 occlusion_break
---SUBDIRS---
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-
doc\eval_baselines\fixtures/clean_single_rally/
total 21
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 .

```

```

drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 ..
-rw-r--r-- 1 avidu 197609   747 Jun 16 11:54 expected.json
-rw-r--r-- 1 avidu 197609    28 Jun 16 11:54 labels.csv
-rw-r--r-- 1 avidu 197609   648 Jun 16 11:54 provenance.json
-rw-r--r-- 1 avidu 197609  3287 Jun 16 11:54 trajectory.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-
doc\eval_baselines\fixtures\multi_rally_with_deadtime/
total 29
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 .
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 ..
-rw-r--r-- 1 avidu 197609  1037 Jun 16 11:54 expected.json
-rw-r--r-- 1 avidu 197609    47 Jun 16 11:54 labels.csv
-rw-r--r-- 1 avidu 197609   665 Jun 16 11:54 provenance.json
-rw-r--r-- 1 avidu 197609  9183 Jun 16 11:54 trajectory.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-
doc\eval_baselines\fixtures\occlusion_break/
total 29
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 .
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 ..
-rw-r--r-- 1 avidu 197609  1038 Jun 16 11:54 expected.json
-rw-r--r-- 1 avidu 197609    47 Jun 16 11:54 labels.csv
-rw-r--r-- 1 avidu 197609   654 Jun 16 11:54 provenance.json
-rw-r--r-- 1 avidu 197609  8823 Jun 16 11:54 trajectory.csv

```

## 20. user (2026-06-16T06:27:31.508Z)

```

1      name: CI
2
3      on:
4        push:
5          branches: [master]
6        pull_request:
7
8      permissions:
9        contents: read
10
11     concurrency:
12       group: ci-${{ github.ref }}
13       cancel-in-progress: true
14
15     jobs:
16       # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17       # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18       # tracked .py, including scripts/ and training/, which the backend-only
19       # py_compile smoke sweep does not reach.
20       lint:
21         name: Ruff lint
22         runs-on: ubuntu-latest
23         steps:
24           - uses: actions/checkout@v6
25           - uses: actions/setup-python@v6
26             with:
27               python-version: "3.13"
28           - name: Install ruff
29             run: python -m pip install ruff
30           - name: Ruff check
31             run: ruff check . --output-format github
32
33       # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34       # per-module quarantine list ? ratchet by removing entries as modules clean
35       # up. Typed core deps are installed so mypy sees their real signatures
36       # (without them ignore_missing_imports would Any-stub half the checks).
37       typecheck:
38         name: Mypy (lenient baseline)
39         runs-on: ubuntu-latest

```

```

40     steps:
41         - uses: actions/checkout@v6
42         - uses: actions/setup-python@v6
43         with:
44             python-version: "3.13"
45         - name: Install mypy + typed core deps
46           run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47         - name: Mypy
48           run: mypy backend training
49
50     # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
51     # module-level import error shipping while the suite stays green.
52     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53     # on machines without the GPU stack (test_import_smoke stubs what's absent).
54     import-smoke:
55         name: Syntax sweep + import smoke (no GPU stack)
56         runs-on: ubuntu-latest
57         steps:
58             - uses: actions/checkout@v6
59             - uses: actions/setup-python@v6
60             with:
61                 python-version: "3.13"
62             - name: Install minimal deps
63               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64             - name: Run import smoke tests
65               run: python -m pytest tests/test_import_smoke.py -v
66
67     full-suite:
68         name: Full test suite (offline, mocked)
69         runs-on: ubuntu-latest
70         steps:
71             - uses: actions/checkout@v6
72             - uses: actions/setup-python@v6
73             with:
74                 python-version: "3.13"
75                 cache: pip
76             - name: System libs for opencv-python
77               run: |
78                 sudo apt-get update
79                 sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
80             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81               # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82               # need an out-of-band index that PEP 621 can't express. Keep installing
83               # them separately here, before the editable install pulls the rest.
84               run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85             - name: Install package (editable) + dev tooling
86               # Editable PEP 621 install: runtime deps + the `dev` group
87               # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88               run: python -m pip install -e .[dev]
89             # obsreport (private, soft dep) is absent here: reporting tests skip via
90             # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91             # is therefore calibrated against THIS lane's number (local runs with
92             # obsreport installed measure a little higher).
93             # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94             # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95             # without an owner decision.
96             - name: Run full suite (with coverage floor)
97               run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
99     # Cross-OS determinism guard for the committed golden regression fixtures

```

```

100 # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
101 # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
102 # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
103 # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
104 # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
105 golden-crossos:
106     name: Golden fixtures cross-OS (${ matrix.os })
107     strategy:
108         fail-fast: false
109         matrix:
110             os: [ubuntu-latest, windows-latest, macos-latest]
111     runs-on: ${ matrix.os }
112     steps:
113         - uses: actions/checkout@v6
114         - uses: actions/setup-python@v6
115         with:
116             python-version: "3.13"
117         - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
118           run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
119         - name: Golden-fixtures characterization (cross-OS determinism guard)
120           run: |
121             python -m backend.eval.golden_fixtures --check
122             python -m pytest tests/test_golden_fixtures.py -q
123

```

## 21. assistant (2026-06-16T06:27:35.306Z)

Let me read the committed fixture files and the README to verify the actual committed content.

## 22. user (2026-06-16T06:27:36.755Z)

```

1      {
2          "candidates": [
3              {
4                  "end": 3.966667,
5                  "shuttle": {
6                      "active_ratio": 1.008475,
7                      "duration": 3.933333,
8                      "mean_velocity": 1.27678,
9                      "net_crossings": 7,
10                     "peak_velocity": 1.970041
11                 },
12                 "start": 0.033333
13             }
14         ],
15         "control_segments": [
16             [
17                 0.033333,
18                 3.966667
19             ]
20         ],
21         "gt_hash": "6c93801fd607",
22         "iou": 0.5,
23         "schema_version": 1,
24         "score": {
25             "f1": 1.0,
26             "false_negatives": 0,
27             "false_positives": 0,
28             "iou_threshold": 0.5,
29             "mean_end_error": 0.033333,
30             "mean_iou": 0.983333,
31             "mean_start_error": 0.033333,
32             "num_gt": 1,
33             "num_pred": 1,

```



```

34         "precision": 1.0,
35         "recall": 1.0,
36         "true_positives": 1
37     },
38     "slug": "clean_single_rally"
39 }
40

```

### 23. user (2026-06-16T06:27:37.226Z)

```

1     {
2         "description": "One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
3         "fps": 30.0,
4         "frame_width": 1280.0,
5         "iou": 0.5,
6         "kind": "synthetic",
7         "license": "synthetic",
8         "minors_free": true,
9         "note": "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only.
Plumbing-correctness fixture ? NOT a measure of rally-detection quality ($6).",
10        "paths": {
11            "expected": "expected.json",
12            "labels": "labels.csv",
13            "trajectory": "trajectory.csv"
14        },
15        "schema_version": 1,
16        "slug": "clean_single_rally",
17        "windowing": {
18            "max_win": 0.0,
19            "mg": 2.0,
20            "mwd": 2.0,
21            "vt": 0.02
22        }
23    }
24

```

### 24. user (2026-06-16T06:27:38.156Z)

```

1     # `eval_baselines/fixtures/` ? Golden Regression Fixtures (Tier-0, synthetic)
2
3     Video-free, **CONTROL-only** characterization fixtures for the rally-segmentation
pipeline.
4     A contributor on **any machine ? no video, GPU, WSL, or DB** ? replays these committed
5     structured files through the already-shipped eval stack and gets a red/green regression
6     signal. See `docs/GOLDEN_REGRESSION_FIXTURES.md` (deliverable **M1**; $4, $6, $7).
7
8     ## What's here
9
10    Each fixture is one directory:
11
12    | File | Purpose |
13    |-----|-----|
14    | `trajectory.csv` | Synthetic shuttle trajectory (`Frame,Visibility,X,Y`) ? the freeze
seam input. |
15    | `labels.csv` | Synthetic ground-truth rally intervals (`start,end`). |
16    | `expected.json` | Frozen replay snapshot (candidates + 5 shuttle features + CONTROL
segments + score). |
17    | `provenance.json` | Pinned windowing params + privacy/licensing tags + description. |
18
19    `manifest.json` lists every fixture; `backend/eval/golden_fixtures.py` is the shared
replay
20    module + freeze/refresh/check CLI.
21
22    ## Guardrails (non-negotiable)

```

```

23
24     - **Synthetic / owned only.** Every fixture is generated by deterministic formulae (NO
random)
25     ? zero third-party footage, zero provenance/licensing risk. Tagged `kind:
"synthetic"`,
26     `license: "synthetic"`, `minors_free: true`.
27     - **No biometric / identity streams.** Tier-0 carries the **5 shuttle feature columns
only**
28     (`duration, peak_velocity, mean_velocity, active_ratio, net_crossings`). There is, by
design,
29     **no person / pose / gait / face / audio / player field anywhere in the schema** ? a
test
30     asserts this positively (deletion-at-source, $4c). Minors and biometrics are excluded.
31     - **CONTROL / pure-stdlib only.** The replay uses `experiment.CONTROL` (`is_learned() ==
False`)
32     ? no numpy / logistic-regression. Learned variants' near-threshold decisions flip
across CPUs,
33     so they can **never** anchor a cross-machine snapshot ($4d / D5).
34     - **Parse-compare, never byte-compare.** The gate `json.load`'s both sides: exact
equality on
35     int/structural fields, abs-tolerance `1e-6` on floats (CRLF/float-format safe).
36     - **Windowing is pinned per fixture.** The `(vt, mg, mwd, max_win)` params live in each
37     `provenance.json` and are passed explicitly to `build_candidates` ? the replay does
**not**
38     depend on the named `windowing.SERVED` constant, so a future SERVED retune can't
silently
39     churn fixtures.
40
41     ## Fixtures are GENERATED, never hand-edited
42
43     Do **not** hand-edit `expected.json` / `provenance.json` / the CSVs. They are produced
44     correct-by-construction by the CLI.
45
46     ```bash
47     # Generate the synthetic trajectory.csv + labels.csv for all built-in scenarios, then
48     # freeze expected.json + provenance.json + (re)write manifest.json:
49     python -m backend.eval.golden_fixtures --freeze-synthetic
50
51     # Replay all fixtures and parse-compare vs the committed expected.json (exit 0/1):
52     python -m backend.eval.golden_fixtures --check
53
54     # Re-freeze expected.json from the COMMITTED trajectory+labels (NO regeneration). ? This
can
55     # rubber-stamp a real regression ? review the visible diff and record a reason ($6):
56     python -m backend.eval.golden_fixtures --refresh-snapshot --all --reason "<why>"
57     python -m backend.eval.golden_fixtures --refresh-snapshot --slug clean_single_rally
--reason "<why>"
58     ```
59
60     ## Honest limits ($6 ? rides in the test pass/fail messages, not only here)
61
62     - A green suite proves only that the **deterministic post-trajectory transforms** are
unchanged
63     for the frozen cases. It does **not** cover the WASB/TrackNet detector, frame
decoding, the
64     AI/provider handoff, chunking, the ffmpeg stitch, DB persistence, or the API filter.
65     - Characterization is **behaviour-locking, not correctness** ? it freezes current
behaviour
66     *including current bugs*. "Passing" = "behaviour unchanged", never "behaviour
correct".
67     - **Synthetic-tier F1 measures plumbing correctness, NOT field quality.** Never cite a
synthetic
68     fixture's F1 in `QUALITY_ITERATIONS.md` or the paper as rally-detection performance.
69

```

## 25. user (2026-06-16T06:27:38.638Z)

```
1      [
2      {
3          "fps": 30.0,
4          "frame_width": 1280.0,
5          "kind": "synthetic",
6          "paths": {
7              "dir": "clean_single_rally",
8              "expected": "clean_single_rally/expected.json",
9              "labels": "clean_single_rally/labels.csv",
10             "provenance": "clean_single_rally/provenance.json",
11             "trajectory": "clean_single_rally/trajectory.csv"
12         },
13         "schema_version": 1,
14         "slug": "clean_single_rally",
15         "windowing": {
16             "max_win": 0.0,
17             "mg": 2.0,
18             "mwd": 2.0,
19             "vt": 0.02
20         }
21     },
22     {
23         "fps": 30.0,
24         "frame_width": 1280.0,
25         "kind": "synthetic",
26         "paths": {
27             "dir": "multi_rally_with_deadtime",
28             "expected": "multi_rally_with_deadtime/expected.json",
29             "labels": "multi_rally_with_deadtime/labels.csv",
30             "provenance": "multi_rally_with_deadtime/provenance.json",
31             "trajectory": "multi_rally_with_deadtime/trajectory.csv"
32         },
33         "schema_version": 1,
34         "slug": "multi_rally_with_deadtime",
35         "windowing": {
36             "max_win": 0.0,
37             "mg": 2.0,
38             "mwd": 2.0,
39             "vt": 0.02
40         }
41     },
42     {
43         "fps": 30.0,
44         "frame_width": 1280.0,
45         "kind": "synthetic",
46         "paths": {
47             "dir": "occlusion_break",
48             "expected": "occlusion_break/expected.json",
49             "labels": "occlusion_break/labels.csv",
50             "provenance": "occlusion_break/provenance.json",
51             "trajectory": "occlusion_break/trajectory.csv"
52         },
53         "schema_version": 1,
54         "slug": "occlusion_break",
55         "windowing": {
56             "max_win": 0.0,
57             "mg": 2.0,
58             "mwd": 2.0,
59             "vt": 0.02
60         }
61     }
62 ]
```

## 26. assistant (2026-06-16T06:27:43.678Z)

Now I have a thorough understanding. Let me focus the privacy/de-attribution lens analysis on the key concerns. Let me trace through the `\_scan\_forbidden` logic carefully and examine the surrogate slug / source-map handling.

Let me verify a few specific things about the scan completeness. First, the key question: does `\_scan\_forbidden` scan the `trajectory.csv` and `labels.csv` (the raw coordinate data), or only `expected.json` and `provenance.json`?

## 27. user (2026-06-16T06:27:45.703Z)

```
501: def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
578:     _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
580:     _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
```

## 28. user (2026-06-16T06:27:46.886Z)

```
482: def _new_surrogate_slug() -> str:
483:     """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` ($4c).
487:     tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives off-
git.
501: def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
522:     for sp in source_paths:
570:         source_note: str = "",
578:         opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by
default), the
583:         NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
610:         # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source name
---
611:         out_slug = slug if slug is not None else _new_surrogate_slug()
663:         "source_note": str(source_note),
666:         "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
677:         source_paths = [str(traj_path), str(labels_path)]
679:             where=f"{out_slug}/expected.json", source_paths=source_paths)
681:             where=f"{out_slug}/provenance.json", source_paths=source_paths)
919:         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
935:         ap.add_argument("--source-note", dest="source_note", default="",
977:             time_origin_reset=args.time_origin_reset, source_note=args.source_note)
```

## 29. assistant (2026-06-16T06:27:51.003Z)

This confirms a critical finding: `\_scan\_forbidden` is only called on `expected.json` and `provenance.json` ? NOT on the written `trajectory.csv` or `labels.csv`. Let me check the `.gitattributes` and whether there's anything about the off-git surrogate map being enforced, and check the gt\_hash leak vector.

## 30. user (2026-06-16T06:27:53.623Z)

```
=== .gitattributes (fixtures) ===
```

**Golden Regression Fixtures (M1) ? pin LF line endings so the committed fixtures are byte-stable across OSes despite core.autocrlf. The characterization gate parse-compa fixtures reproducible. See docs/GOLDEN\_REGRESSION\_FIXTURES.md §4d/§4e.**

```
eval_baselines/fixtures/** text eol=lf
=== .gitignore for surrogate map ===
=== labels.csv content ===
start,end
0.000000,4.000000
```

### 31. assistant (2026-06-16T06:28:00.077Z)

Now let me verify the `gt_hash` behavior ? it's computed from the reset GTs and written into both `provenance.json` and `expected.json`. The question is whether `gt_hash` leaks anything. It's a SHA1 of rounded intervals. After time-origin reset, the intervals are reset, so the hash is over reset values. That's fine. But let me confirm: in ``freeze_real_fixture``, the `gt_hash` in `provenance` uses ``reset_gts`` (line 662), and `replay_fixture` computes `gt_hash` from the reset `labels.csv`. Consistent.

Now let me check a subtle determinism concern: the float comparison in ``_scan_forbidden`` and whether a 32-hex MD5 could appear by coincidence, and check the ``is not True`` gate robustness against truthy values. Let me also examine whether the ``gt_hash`` (a 12-hex SHA1) could trip the MD5 regex falsely.

### 32. user (2026-06-16T06:28:05.752Z)

```
fx_slug match: False
gt_hash 12hex match: False
two 16hex adjacent (32): True
```

### 33. assistant (2026-06-16T06:28:10.087Z)

Now let me verify the critical concern about the raw trajectory coordinates (the de Montjoye fingerprinting risk). The doc §4c says "Raw-trajectory exclusion ? never commit the per-frame (Frame,Visibility,X,Y) CSV." But ``freeze_real_fixture`` writes ``trajectory.csv`` with the full per-frame coordinates (lines 635, via ``write_trajectory_csv``). Let me confirm what gets committed and check the doc claim vs. code reality.

### 34. user (2026-06-16T06:28:11.441Z)

```
1      Frame,Visibility,X,Y
2      0,1,640.000000,360.000000
3      1,1,723.164676,372.202099
4      2,1,802.694657,382.294345
5      3,1,875.114101,388.531695
6      4,1,937.257930,389.835657
7      5,1,986.410162,385.980762
8      6,1,1020.422607,377.633558
```

### 35. assistant (2026-06-16T06:28:17.048Z)

This confirms a major architectural contradiction. The committed fixture directory contains ``trajectory.csv`` with full per-frame ``(Frame, Visibility, X, Y)`` coordinates ? and ``freeze_real_fixture`` writes exactly this same raw per-frame CSV for real fixtures (line 635). This directly contradicts the doc's §4c "Raw-trajectory exclusion ? never commit the per-frame ``(Frame,Visibility,X,Y)`` CSV" and TM5's claim of "only ~5 invariant scalars/window."

Let me verify the ``is not True`` gate robustness and check whether the manifest entry for `cleared_real` fixtures could leak the ``label`` or ``source_note``. Let me look at ``_manifest_entry_for`` and ``_append_manifest_entry`` ? the manifest is NOT scanned by ``_scan_forbidden``.

### 36. user (2026-06-16T06:28:19.480Z)

```
minors_free=True      -> blocked=False
minors_free=1         -> blocked=True
minors_free=1.0       -> blocked=True
minors_free='yes'     -> blocked=True
minors_free=[1]       -> blocked=True
minors_free='True'    -> blocked=True
```

### 37. assistant (2026-06-16T06:28:30.676Z)

Good ? the ``is not True`` gate is robust against truthy-but-not-True values (only the literal

`True` passes). That defends against the truthy bypass concern. Now let me check the manifest leak vector: `\_manifest\_entry\_for` only copies slug/fps/frame\_width/windowing/kind/schema\_version/paths ? it does NOT copy `label` or `source\_note`. So the manifest can't leak those. But the manifest is never scanned. Let me confirm the manifest fields can't carry a leak.

The `\_manifest\_entry\_for` derives only structural fields from prov, and `slug` is the surrogate. So no leak via manifest as written.

Now, the most subtle finding ? let me examine the `\_scan\_forbidden` source-path token matching for the false-negative case (where a real identifying token could be MISSED). The scan checks `sp`, `p.name`, `p.stem`. But what about a parent directory name that identifies a player/venue? E.g., source path `C:\videos\Lin\_Dan\_vs\_Lee\match.csv` ? `p.stem` = "match", `p.name` = "match.csv", `sp` = full path. The full path `sp` IS scanned, so "Lin\_Dan" within the full path would be caught IF it appears in the output. But the output (expected.json/provenance.json) never echoes the source path at all in the safe path ? so this is a guard against regression, as documented. Let me verify there's no path echo.

Let me also check one real concern: the `source\_note` is written into provenance.json (line 663) and THEN scanned. But the scan only catches FORBIDDEN\_KEYS, MD5, and source-path tokens. A `source\_note` containing a player name like "Lin Dan semifinal" that is NOT a file path and NOT a forbidden key would pass the scan and be committed. Let me confirm.

### 38. user (2026-06-16T06:28:35.217Z)

FORBIDDEN\_KEYS hit: None

MD5 hit: False

=> player/venue/match-name in source\_note or label: PASSES the scan, gets COMMITTED

### 39. assistant (2026-06-16T06:28:44.251Z)

Confirmed. Now let me check one more determinism/privacy concern: the `fps`/`frame\_width` quasi-identifier. The doc §4c says "Quantize fps/frame\_width ? red-team: at n=6 the raw values are a camera/venue quasi-identifier" and §7 ? admits "fps/fw retained as scale constants; quantize is a P3 option." So this is documented as deferred. Let me confirm the code writes raw fps/frame\_width into both provenance and manifest (not quantized).

The provenance writes `\_round(fps)` and `\_round(frame\_width)` (lines 647-648) ? that's just 6dp rounding, not venue-quantization. And these are exposed in `expected.json` ? No ? `expected.json` (replay\_fixture) does not include fps/frame\_width. But provenance.json and manifest.json do. So the camera/venue quasi-identifier IS committed in provenance. This matches the doc's honest admission (deferred to P3), so it's a documented gap, not a hidden one.

Now let me check the surrogate?source map: the code generates a random slug but never persists any map. The docstring says "The surrogate?source map (if kept) lives off-git." There's no code that writes a map ? so reversibility depends entirely on whether the operator keeps one externally. There's no enforcement, but also no on-git artifact. Let me verify there's truly no map written anywhere.

### 40. user (2026-06-16T06:28:46.275Z)

487: tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives off-git.

610: # --- (b) opaque RANDOM surrogate slug ? never the video\_id MD5, never the source name  
---

679: where=f"{out\_slug}/expected.json", source\_paths=source\_paths)

681: where=f"{out\_slug}/provenance.json", source\_paths=source\_paths)

### 41. assistant (2026-06-16T06:28:55.776Z)

No map is written by the code ? the random slug is genuinely non-reversible (16 bytes of `os.urandom` entropy, no algebraic link to source). That part of the design is sound: the slug itself leaks nothing, and there's no on-git surrogate?source map. The operator-kept map is explicitly off-code/off-git by design.

Now let me verify one final determinism concern in the float scan and `os.urandom` ? the surrogate slug is non-deterministic, which means re-running `freeze\_real\_fixture` without an explicit `--slug` produces a NEW directory each time (orphaning the old). Not a privacy hole but worth noting. More importantly, let me check: does the `\_scan\_forbidden` run BEFORE or AFTER the files are written? It runs AFTER (lines 678-681), so if it raises, the leaked `trajectory.csv`/`labels.csv`/`expected.json`/`provenance.json` are ALREADY on disk and the manifest entry was NOT yet appended (line 684 is after the scan). Let me confirm the ordering creates an orphaned-leak-on-disk situation.

#### 42. user (2026-06-16T06:28:56.921Z)

```
671         _write_json(fdir / "provenance.json", prov)
672
673         expected = replay_fixture(fdir)          # REUSE M1 ? correct-by-construction
snapshot
674         _write_json(fdir / "expected.json", expected)
675
676         # --- (f) POSITIVE privacy assertion AFTER writing (scan the written text, $4c / ?)
-----
677         source_paths = [str(traj_path), str(labels_path)]
678         _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
679                        where=f"{out_slug}/expected.json", source_paths=source_paths)
680         _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
681                        where=f"{out_slug}/provenance.json", source_paths=source_paths)
682
683         # --- (g) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
684         _append_manifest_entry(prov, fixtures_dir)
685         return expected
```

#### 43. assistant (2026-06-16T06:29:05.364Z)

Confirmed. The scan runs after all four files are written to disk. When the scan raises `RefusalError`, the function aborts but leaves `trajectory.csv`, `labels.csv`, `expected.json`, and `provenance.json` (the latter potentially containing the leaked `source\_note`) on disk in `fdir`. No cleanup. The test `test\_positive\_privacy\_assertion\_catches\_leak\_in\_source\_note` only asserts that `RefusalError` is raised ? it does NOT assert the leaked files were removed. Contrast with the synthetic refusal-gate test (`test\_refusal\_gate\_blocks\_unsafe`), which DOES assert "Nothing was written for a refused freeze" (because that gate fires before any write). The post-write scan has no such guarantee.

Let me confirm this gap is real by checking that the test never checks for cleanup, and confirm the gate-vs-scan asymmetry.

#### 44. user (2026-06-16T06:29:14.105Z)

```
RefusalError raised (good)
fdir exists after refusal: True
LEAKED FILES LEFT ON DISK: ['expected.json', 'labels.csv', 'provenance.json', 'trajectory.csv']
source path present in left-behind provenance.json: False
```

#### 45. assistant (2026-06-16T06:29:20.078Z)

Interesting ? the leaked path is NOT in the left-behind provenance.json. Let me check why. The `source\_note=str(traj)` ? `str(traj)` is the absolute path. Let me look at what the scan caught and why the path isn't in the file. Actually wait ? the path `str(traj)` ends in `src\_trajectory.csv`, and `p.stem` = `src\_trajectory`. The token `src\_trajectory` ? is it a substring of a schema literal? `trajectory` IS a schema literal, but `src\_trajectory` is not a substring of `trajectory.csv`. Let me re-examine ? the scan caught the FULL path token `sp`. Let me verify what's actually written and re-run more carefully.

#### 46. user (2026-06-16T06:29:28.420Z)

FROZEN OK (no refusal) -> slug fx\_realleak

source\_note in committed provenance: True  
label in committed provenance: True

#### 47. assistant (2026-06-16T06:29:39.391Z)

This confirms the central `\_scan\_forbidden` incompleteness: a free-text identifying `source\_note` (player names "Lin Dan vs Lee", venue "India Open 2024", "Court 1") AND the `label` field ("mens singles final") pass the scan and are committed verbatim into `provenance.json`. The scan only catches biometric keys, 32-hex MD5, and source-PATH tokens ? it has zero defense against free-text identifiers in `source\_note`/`label`. The earlier test "passed" only because `str(traj)` happened to be a path token the scan does catch.

Now let me verify the de Montjoye fingerprinting concern definitively ? the raw per-frame trajectory IS committed for real fixtures, and the scan does nothing about coordinate fingerprinting. Let me also double check: does `label` get scanned at all? It's in provenance.json which IS scanned, but only for the 3 categories. Confirmed above.

Let me now check whether the `--label` default "cleared\_real" and check the round-trip for the raw trajectory commit one more time ? confirming `freeze\_real\_fixture` writes the full per-frame CSV (not aggregated).

The code at line 635 `write\_trajectory\_csv(rows, fdir / "trajectory.csv")` writes every `(Frame,Visibility,X,Y)` row from `\_reset\_trajectory\_rows(points, t0\_frame)` ? i.e., the complete per-frame trajectory. This is committed to the tracked `eval\_baselines/fixtures/` directory.

I now have all findings. Let me do a final check on the de Montjoye angle: confirm that only the time origin is shifted, X/Y coordinates pass through completely untouched.

#### 48. user (2026-06-16T06:29:40.376Z)

```
538     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
539                               ) -> List[Tuple[int, int, float, float]]:
540         """Shift every `TrajectoryPoint` frame by `-t0_frame` ?
541         `(Frame,Visibility,X,Y)` rows.
542         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
543         invariant under a
544         pure time shift); only the absolute frame index moves toward 0, severing the match
545         timeline.
546         """
547         rows: List[Tuple[int, int, float, float]] = []
548         for p in points:
549             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
550                         float(p.y)))
551         rows.sort(key=lambda r: r[0])
552         return rows
```

#### 49. assistant (2026-06-16T06:29:44.746Z)

Confirmed: X/Y coordinates pass through completely untouched. Only the frame index is shifted. So the raw per-frame `(X,Y)` shuttle trajectory ? the exact fingerprint de Montjoye describes (4 points ? 95% re-ID) ? is committed verbatim for a `cleared\_real` fixture. The time-origin reset severs the absolute-timeline correlation but does nothing to the spatial fingerprint.

I have completed the audit. Let me compile the findings.

#### 50. user (2026-06-16T06:30:41.530Z)

Structured output provided successfully

#### 51. assistant (2026-06-16T06:30:51.727Z)

I've completed the privacy/de-attribution audit of the Golden Regression Fixtures feature.



Here's the summary of what the per-PR verification missed:

**\*\*Two high-severity gaps:\*\***

1. **\*\*`\_scan\_forbidden` is incomplete for free-text identifiers.\*\*** It catches biometric key-substrings, 32-hex MD5s, and source-*\*path\** tokens ? but has zero defense against player/venue/match names typed into the ``source_note`` and ``label`` fields, which are written verbatim into the committed ``provenance.json``. I proved that ``label='mens singles final'`` + ``source_note='Lin Dan vs Lee, India Open 2024, Court 1'`` freezes with no refusal and commits both. The existing test only catches a leak because it passes a *\*path\** string, masking the free-text hole.

2. **\*\*The raw per-frame ``(x,y)`` trajectory is committed verbatim\*\*** for real fixtures ? X/Y pass through untouched in ``_reset_trajectory_rows``, only the frame index shifts. This is the exact de Montjoye spatial fingerprint, and it directly contradicts the doc's §4c "never commit the per-frame CSV" and TM5's "only ~5 invariant scalars/window." The owner's risk acceptance was made against a doc that overstates the de-attribution actually implemented.

**\*\*Two lower-severity gaps:\*\*** the post-write scan leaves leaked files on disk on refusal (no cleanup, unlike the pre-write M2 gate), and the scan never inspects the ``trajectory.csv`/`labels.csv`` it writes.

**\*\*What is genuinely sound\*\*** (verified, not just asserted): the ``is not True`` refusal gate resists truthy-but-not-True bypass (only literal ``True`` passes); the ``fx_<16hex>`` slug is ``os.urandom``-derived with no algebraic link to the source; no surrogate?source map is written on-git; the `gt_hash/slug` shapes don't false-trip the MD5 regex; and the time-origin reset is correctly metric-invariant.

The synthetic fixtures shipped today carry no real data, so these gaps don't block what's committed ? but they block the P3 real-data path the refusal gate exists to enable.